

Introduction to Machine Learning with Python



MACHINE
LEARNING

Dr. Ei Ei Thu

Associate Professor

Faculty of Computer Science

Outlines



1

Why Machine Learning?

2

Why Python?
Python Installation Steps
scikit-learn
Essential Libraries and Tools

3

A First Application: Classifying Iris Species



Introduction



- ❖ Machine learning is about extracting knowledge from data.
- ❖ It is a research field at the intersection of statistics, artificial intelligence, and computer science and is also known as predictive analytics or statistical learning.
- ❖ The application of machine learning methods has in recent years become ubiquitous in everyday life.
- ❖ From automatic recommendations of which movies to watch, to what food to order or which products to buy, to personalized online radio and recognizing your friends in your photos, many modern websites and devices have machine learning algorithms at their core.
- ❖ A complex website like Facebook, Amazon, or Netflix, it is very likely that every part of the site contains multiple machine learning models.

Why Machine Learning?

- ❖ In the early days of “intelligent” applications, many systems used hand coded rules of “if” and “else” decisions to process data or adjust to user input. Using hand coded rules to make decisions has two major disadvantages:
 - ❖ The logic required to make a decision is specific to a single domain and task. Changing the task even slightly might require a rewrite of the whole system.
 - ❖ Designing rules requires a deep understanding of how a decision should be made by a human expert.
- ❖ Hand coded approach will fail in detecting faces in images. The main problem is that the way in which pixels are “perceived” by the computer is very different from how humans perceive a face.
- ❖ By using machine learning, simply presenting a program with a large collection of images of faces is enough for an algorithm to determine what characteristics are needed to identify a face.

Problems Machine Learning Can Solve

- ❖ The most successful kinds of machine learning algorithms are those that automate decision-making processes by generalizing from known examples, which is known as **supervised learning**, the user provides the algorithm with pairs of inputs and desired outputs, and the algorithm finds a way to produce the desired output given an input.
- ❖ **Example:** Spam classification, using machine learning, the user provides the algorithm with a large number of emails (which are input), together with information about whether any of these emails are spam (which is the desired output). Given a new email, the algorithm will then produce a prediction as to whether the new email is spam.

Examples of supervised machine learning tasks

- ❖ Identifying the zip code from handwritten digits on an envelope
- ❖ Determining whether a tumor is benign based on a medical image
- ❖ Detecting fraudulent activity in credit card transactions

Unsupervised machine learning tasks

- ❖ In unsupervised learning, only the input data is known, and no known output data is given to the algorithm.
- ❖ Example of unsupervised learning included:
 - ❖ Identifying topics in a set of blog posts
 - ❖ Detecting abnormal access patterns to a website

- ❖ For both supervised and unsupervised learning tasks, it is important to have a **representation of input data** that a computer can understand.

Knowing Your Task and Knowing Your Data



- ❖ What question(s) am I trying to answer? Do I think the data collected can answer that question?
 - ❖ What is the best way to phrase my question(s) as a machine learning problem?
 - ❖ Have I collected enough data to represent the problem I want to solve?
 - ❖ What features of the data did I extract, and will these enable the right predictions?
 - ❖ How will I measure success in my application?
 - ❖ How will the machine learning solution interact with other parts of my research or business product?
- ❖ Each entity or **row** is known as **sample** (or data point) in machine learning, while the **columns** are called **features**.

Why Python?

- ❖ Python combines the power of general-purpose programming languages with the ease of use of domain-specific scripting languages like MATLAB or R.
- ❖ Python has libraries for data loading, visualization, statistics, natural language processing, image processing, and more.
- ❖ Python also allows for the creation of complex graphical user interfaces (GUIs) and web services, and for integration into existing systems.

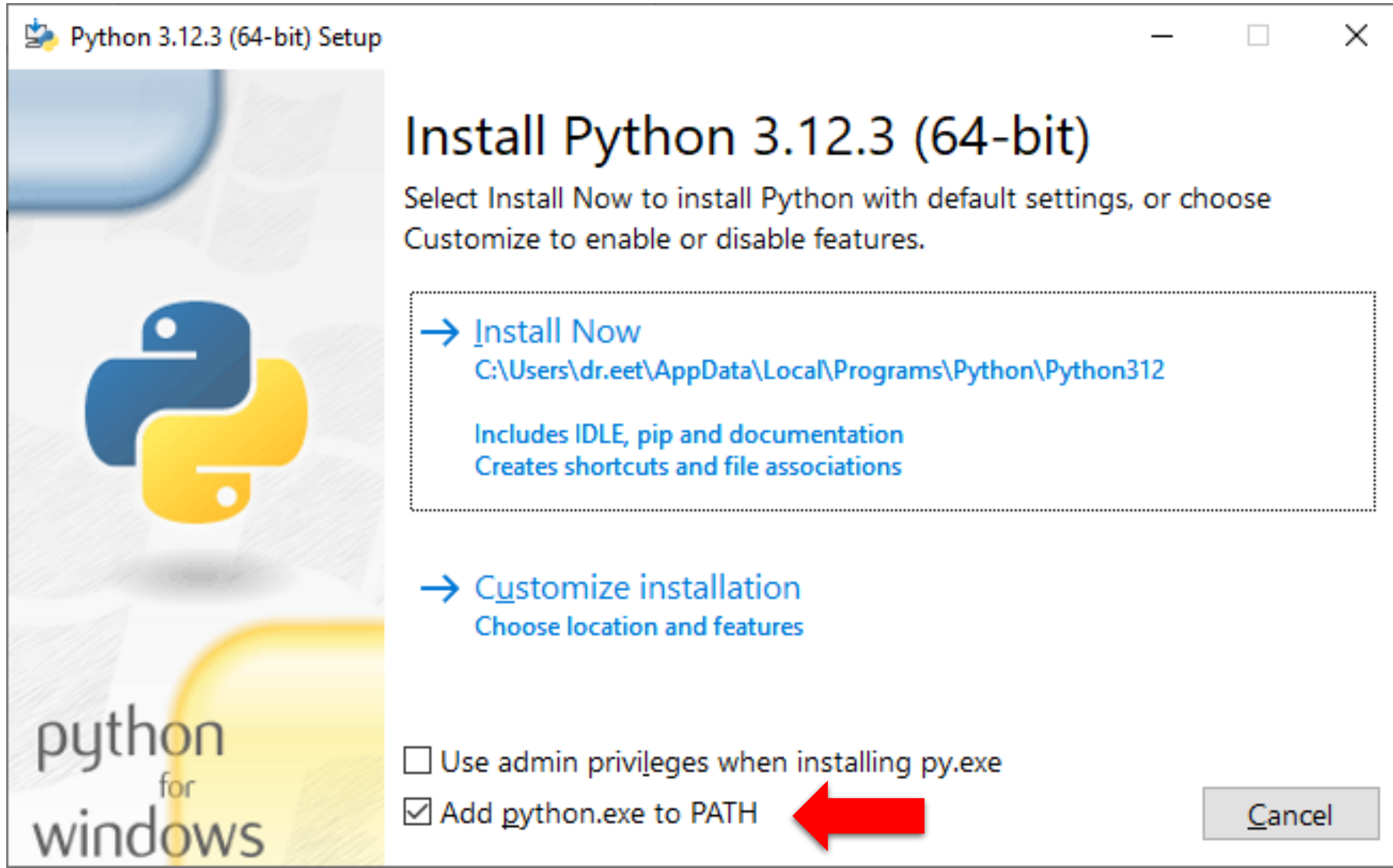


Configuring Jupyter notebook for Python

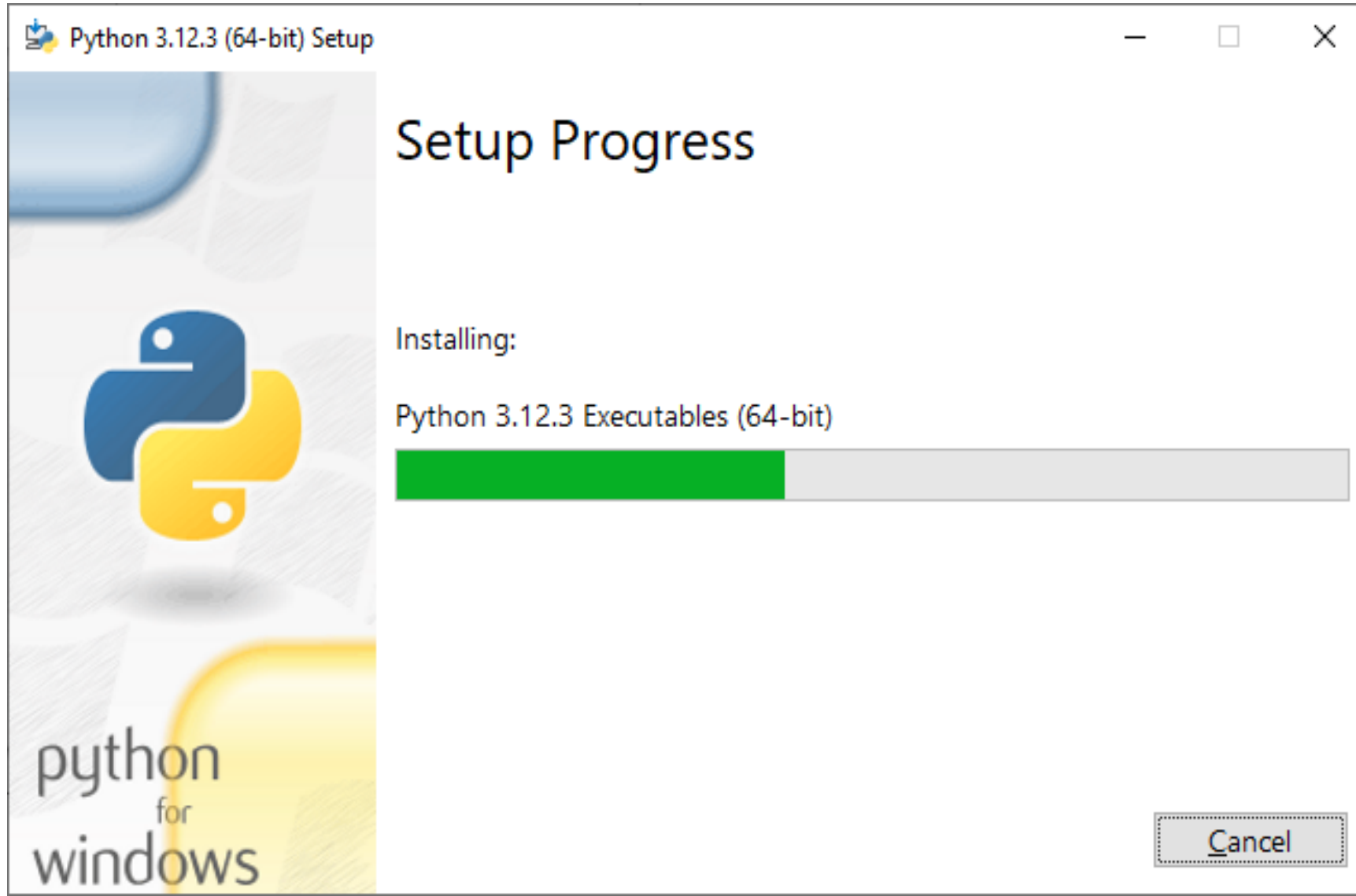


Step 1: Install Python

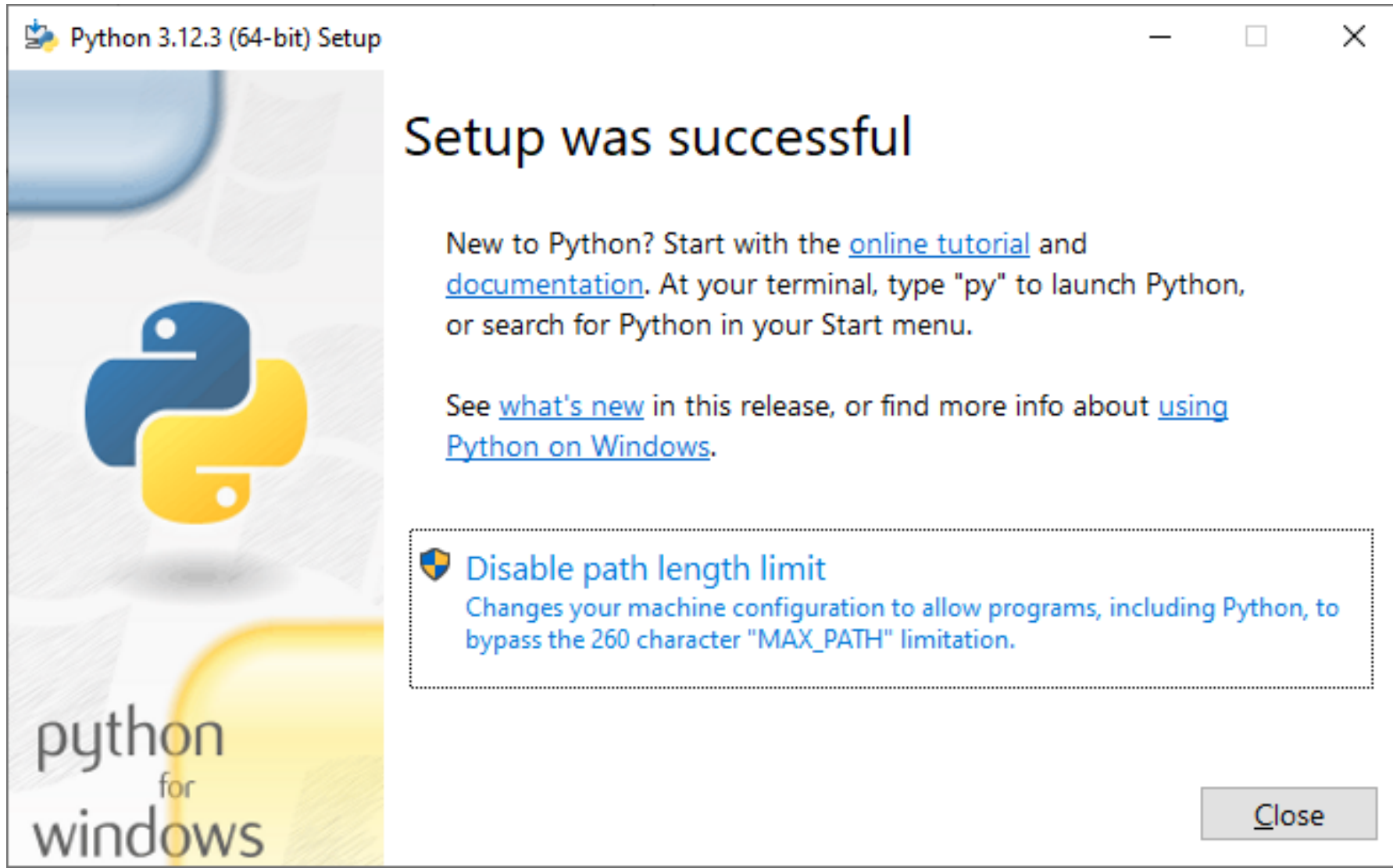
<https://www.python.org/downloads/>



Step 1: Install Python -> Setup Progress



Step 1: Install Python -> Setup was Successful



Step 2: Install Jupyter Notebook via PIP



```
Command Prompt

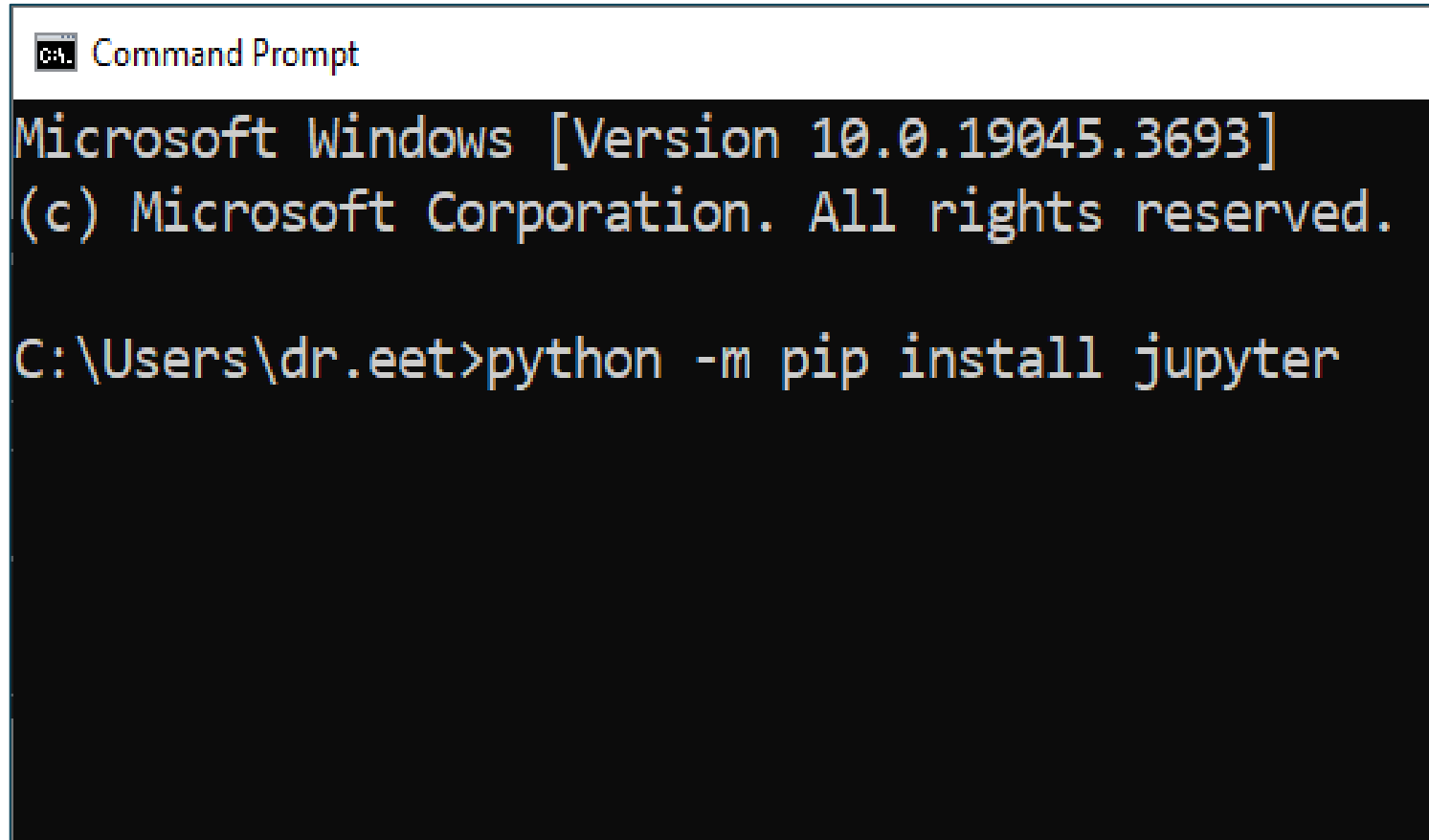
C:\Users\dr.eet>python --version
Python 3.12.3

C:\Users\dr.eet>
```

1. Open Command Prompt by searching 'cmd' in the Windows search bar.
2. Check if Python is installed by running the 'python --version' command.

Step 2: Install Jupyter Notebook via PIP

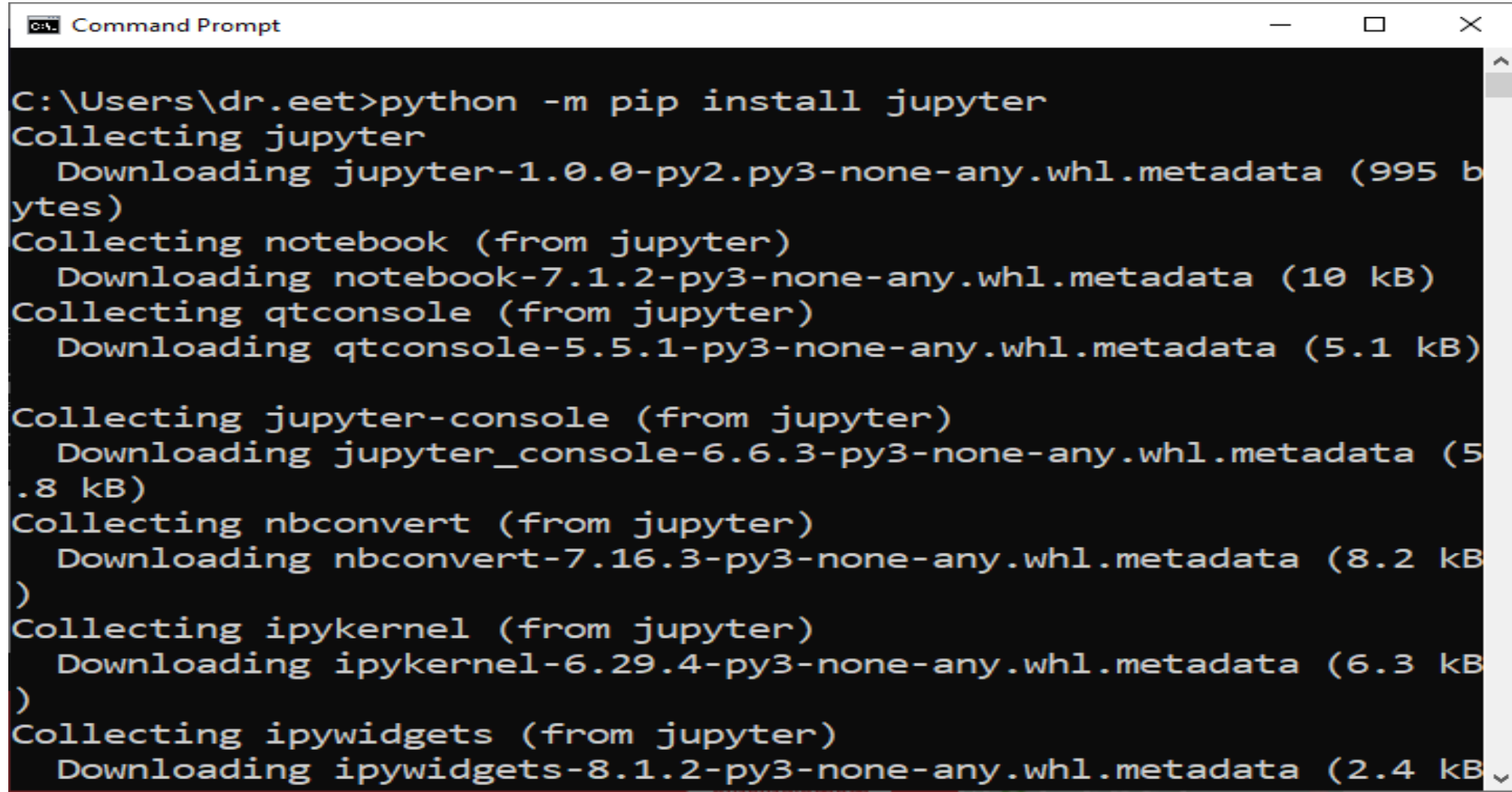
1. To install Jupyter Notebook via pip, run the command
'**python -m pip install jupyter**'

A screenshot of a Windows Command Prompt window. The title bar says 'C:\ Command Prompt'. The text inside shows the Windows version and copyright information, followed by the command 'python -m pip install jupyter' being entered at the prompt.

```
C:\ Command Prompt  
Microsoft Windows [Version 10.0.19045.3693]  
(c) Microsoft Corporation. All rights reserved.  
C:\Users\dr.eet>python -m pip install jupyter
```

Step 2: Install Jupyter Notebook -> Installation Progress

1. Please wait to complete the installation process

A screenshot of a Windows Command Prompt window titled 'Command Prompt'. The window has a black background with white text. The text shows the command 'python -m pip install jupyter' being executed. The output shows the progress of installing Jupyter and its dependencies: jupyter, notebook, qtconsole, jupyter-console, nbconvert, ipykernel, and ipywidgets. Each package is shown with its version, platform, and the size of the metadata file being downloaded.

```
C:\Users\dr.eet>python -m pip install jupyter
Collecting jupyter
  Downloading jupyter-1.0.0-py2.py3-none-any.whl.metadata (995 b
ytes)
Collecting notebook (from jupyter)
  Downloading notebook-7.1.2-py3-none-any.whl.metadata (10 kB)
Collecting qtconsole (from jupyter)
  Downloading qtconsole-5.5.1-py3-none-any.whl.metadata (5.1 kB)

Collecting jupyter-console (from jupyter)
  Downloading jupyter_console-6.6.3-py3-none-any.whl.metadata (5
.8 kB)
Collecting nbconvert (from jupyter)
  Downloading nbconvert-7.16.3-py3-none-any.whl.metadata (8.2 kB
)
Collecting ipykernel (from jupyter)
  Downloading ipykernel-6.29.4-py3-none-any.whl.metadata (6.3 kB
)
Collecting ipywidgets (from jupyter)
  Downloading ipywidgets-8.1.2-py3-none-any.whl.metadata (2.4 kB
```

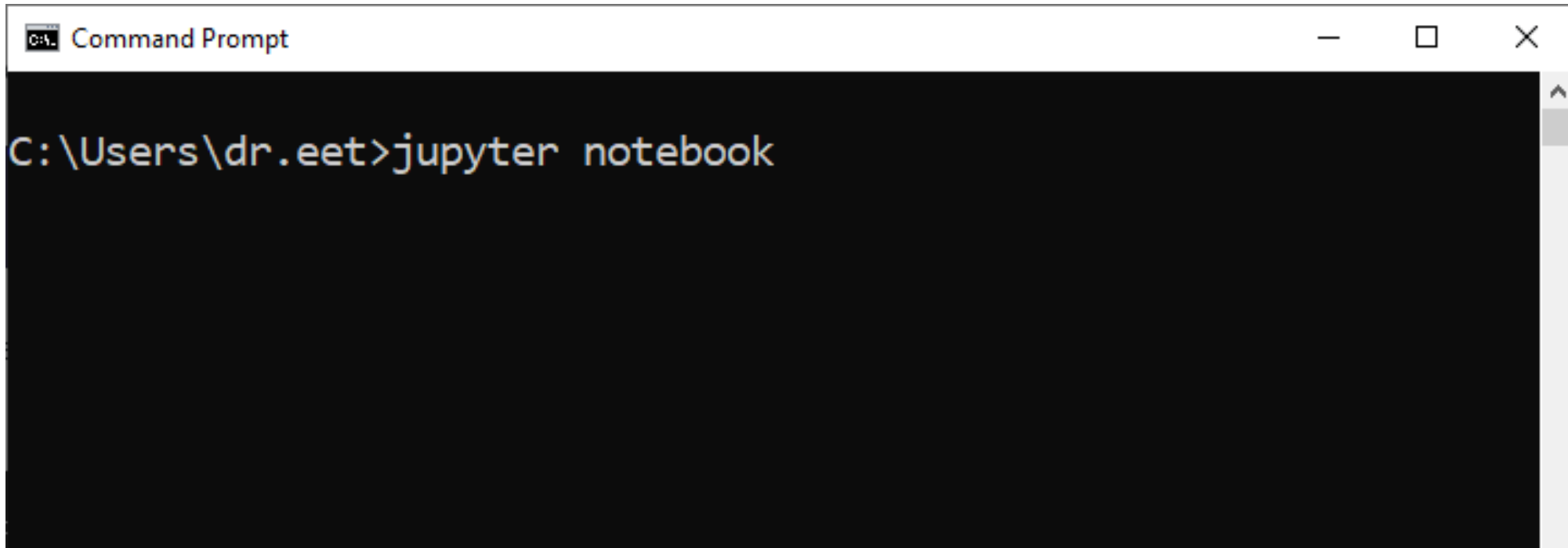
Step 2: Install Jupyter Notebook -> Installation Success

```
Command Prompt
une-3.0.2 nbclient-0.10.0 nbconvert-7.16.3 nbformat-5.10.4 nest-
asyncio-1.6.0 notebook-7.1.2 notebook-shim-0.2.4 overrides-7.7.0
packaging-24.0 pandocfilters-1.5.1 parso-0.8.4 platformdirs-4.2
.0 prometheus-client-0.20.0 prompt-toolkit-3.0.43 psutil-5.9.8 p
ure-eval-0.2.2 pycparser-2.22 pygments-2.17.2 python-dateutil-2.
9.0.post0 python-json-logger-2.0.7 pywin32-306 pywinpty-2.0.13 p
yyaml-6.0.1 pyzmq-26.0.0 qtconsole-5.5.1 qtpy-2.4.1 referencing-
0.34.0 requests-2.31.0 rfc3339-validator-0.1.4 rfc3986-validator
-0.1.1 rpds-py-0.18.0 send2trash-1.8.3 six-1.16.0 sniffio-1.3.1
soupsieve-2.5 stack-data-0.6.3 terminado-0.18.1 tinycss2-1.2.1 t
ornado-6.4 traitlets-5.14.2 types-python-dateutil-2.9.0.20240316
uri-template-1.3.0 urllib3-2.2.1 wcwidth-0.2.13 webcolors-1.13
webencodings-0.5.1 websocket-client-1.7.0 widgetsnbextension-4.0
.10

C:\Users\dr.eet>
```

Step 3: Access Jupyter Notebook

To access Jupyter Notebook via command prompt, run the command
‘jupyter notebook’

A screenshot of a Windows Command Prompt window. The title bar reads 'C:\> Command Prompt'. The command prompt shows the path 'C:\Users\dr.eet>' followed by the command 'jupyter notebook' entered in white text on a black background.

နောက်တစ်ခု Jupyter Notebook ပြန်ဖွင့်ချင်တိုင်း Command Prompt ကို ဖွင့်ပြီး ဒီနေရာကနေပြန်စရပါမယ်
ရှေ့က Installation Process တွေ ပြန်လုပ်စရာမလိုပါ

The Jupyter Notebook will open via Browser.

A screenshot of a web browser window displaying the Jupyter Notebook interface. The browser's address bar shows 'localhost:8888/tree'. The page header includes the Jupyter logo and a navigation menu with 'File', 'View', 'Settings', and 'Help'. Below the header, there are two tabs: 'Files' and 'Running'. The 'Files' tab is active, showing a file explorer view. At the top of the file explorer, there is a prompt 'Select items to perform actions on them.' and three buttons: 'New', 'Upload', and a refresh icon. Below this, a list of files and folders is displayed in a table format. The table has three columns: 'Name', 'Last Modified', and 'File Size'. The 'Name' column includes checkboxes for each item. The 'Last Modified' column shows the time since the item was last modified. The 'File Size' column is currently empty. The items listed are: 3D Objects, Contacts, Desktop, Documents, Downloads, Favorites, Links, Music, OneDrive, Pictures, Saved Games, Searches, and Videos. Most items were last modified '4 months ago', while 'Downloads' was modified '21 minutes ago' and 'Desktop' was modified '20 days ago'.

☐ Name	Last Modified	File Size
☐ 3D Objects	4 months ago	
☐ Contacts	4 months ago	
☐ Desktop	20 days ago	
☐ Documents	29 days ago	
☐ Downloads	21 minutes ago	
☐ Favorites	4 months ago	
☐ Links	4 months ago	
☐ Music	4 months ago	
☐ OneDrive	4 months ago	
☐ Pictures	4 months ago	
☐ Saved Games	4 months ago	
☐ Searches	4 months ago	
☐ Videos	4 months ago	

You can access the Python program file via Jupyter Notebook Workspace



The image shows a Windows File Explorer window on the left and a Jupyter Notebook interface on the right. A red arrow points from the File Explorer's address bar to the Jupyter interface.

File Explorer (Left): The address bar shows the path: `This PC > Local Disk (C:) > Users > dr.eet`. The left sidebar shows the navigation pane with various folders. The main pane displays a list of files and folders with columns for Name and Date modified.

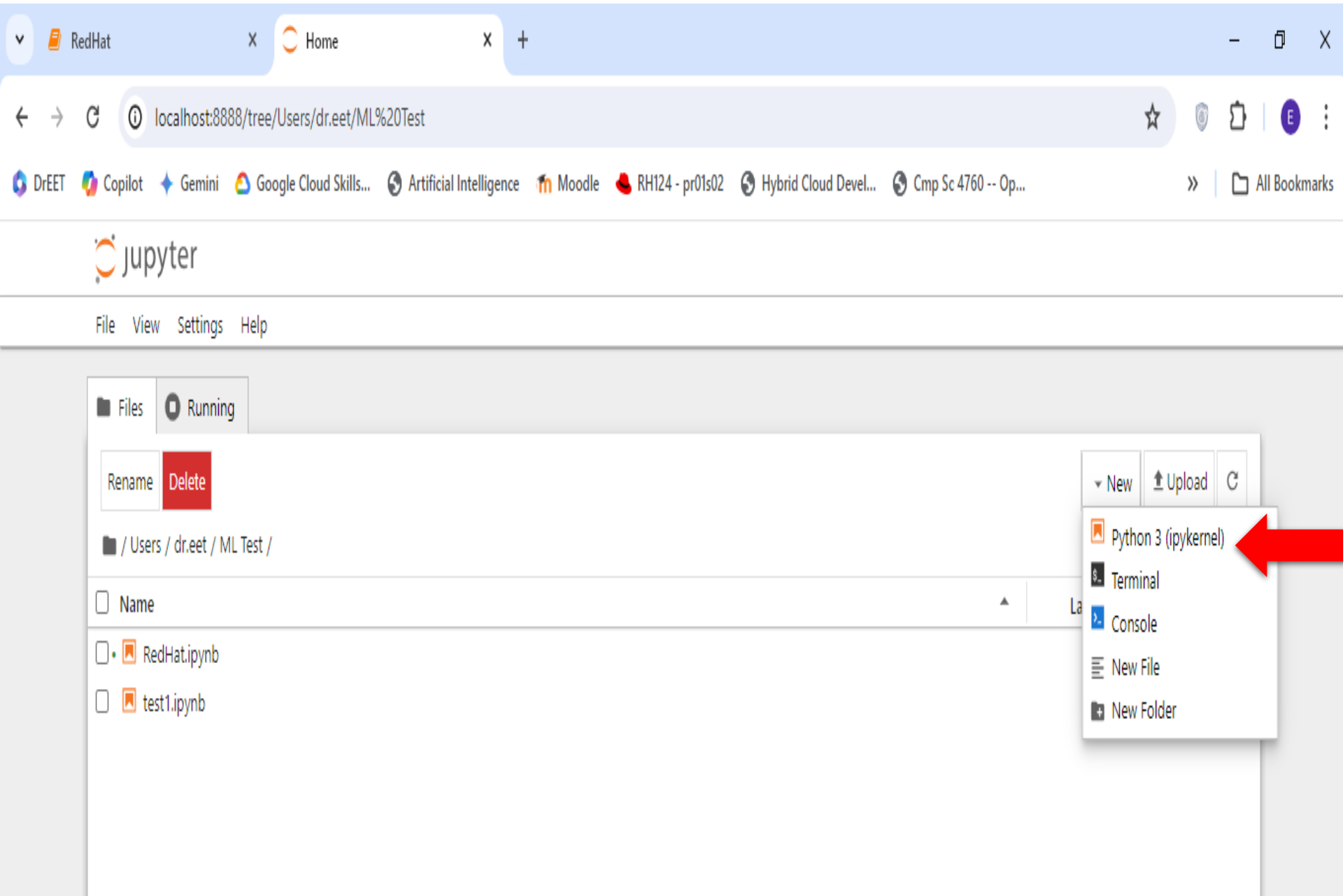
Jupyter Notebook (Right): The browser address bar shows `localhost:8888/tree`. The Jupyter interface includes a menu bar (File, View, Settings, Help) and a toolbar with buttons for File, Running, Rename, and Delete. Below the toolbar is a table listing files and folders.

Name	Last Modified	File Size
3D Objects	4 months ago	
Contacts	4 months ago	
Desktop	20 days ago	
Documents	29 days ago	
Downloads	24 minutes ago	
Favorites	4 months ago	
Links	4 months ago	
Music	4 months ago	
OneDrive	4 months ago	
Pictures	4 months ago	
Saved Games	4 months ago	
Searches	4 months ago	
Videos	4 months ago	

Create a new folder to store Python programs.



A screenshot of the JupyterLab web interface. The browser's address bar shows 'localhost:8888/tree/Users/dr.eet'. The JupyterLab header includes the 'jupyter' logo and a menu bar with 'File', 'View', 'Settings', and 'Help'. Below the header, there are two tabs: 'Files' and 'Running'. The 'Files' tab is active, showing a file browser view of the directory '/ Users / dr.eet /'. A list of files and folders is displayed, including '3D Objects', 'Contacts', 'Desktop', 'Documents', 'Downloads', 'Favorites', 'Links', 'ML Test', 'Music', 'OneDrive', 'Pictures', 'Saved Games', 'Searches', and 'Videos'. On the right side of the file browser, there is a 'New' button. A dropdown menu is open from this button, showing options: 'Python 3 (ipykernel)', 'Terminal', 'Console', 'New File', and 'New Folder'. A red arrow points to the 'New Folder' option. Below these options, a list of recent files and folders is shown, including 'yesterday', 'last month', and several entries with timestamps like '18 hours ago', '5 months ago', and '58 minutes ago'.



RedHat Home

localhost:8888/tree/Users/dr.eet/ML%20Test

DrEET Copilot Gemini Google Cloud Skills... Artificial Intelligence Moodle RH124 - pr01s02 Hybrid Cloud Devel... Cmp Sc 4760 -- Op...

jupyter

File View Settings Help

Files Running

Rename Delete

/ Users / dr.eet / ML Test /

Name

RedHat.ipynb

test1.ipynb

New Upload

Python 3 (ipykernel)

Terminal

Console

New File

New Folder

Create a new Python file from Python 3

You can easily write and run Python code in Jupyter Notebook.



The screenshot shows a web browser window with the Jupyter Notebook interface. The browser tabs include 'AD141 - ch02', 'Home', and 'RedHat'. The address bar shows 'localhost:8888/notebooks/ML%20Test/RedHat.ipynb'. The Jupyter header shows 'jupyter RedHat Last Checkpoint: 6 hours ago' and a 'Trusted' badge. The menu bar includes 'File', 'Edit', 'View', 'Run', 'Kernel', 'Settings', and 'Help'. The toolbar includes icons for saving, opening, and running code. The main area shows a code cell with the following Python code:

```
[3]: num1=int(input("Enter num1"))
      num2=int(input("Enter num1"))
      for num in range(num1,num2):
          print(num)
```

Below the code cell, the output is displayed:

```
Enter num1 10
Enter num1 15
10
11
12
13
14
```

Two red arrows point to the code and output areas. The first arrow points to the code cell with the text 'Write code here.' The second arrow points to the output cell with the text 'Running Results'.

scikit-learn

- ❖ scikit-learn contains a number of state-of-the-art machine learning algorithms, as well as comprehensive documentation about each algorithm.
- ❖ scikit-learn is a very popular tool, and the most prominent Python library for machine learning.
- ❖ scikit-learn depends on two other Python packages, NumPy and SciPy.
- ❖ For plotting and interactive development, install matplotlib, IPython, and the Jupyter Notebook.

Installing scikit-learn



```
pip install numpy scipy matplotlib ipython scikit-learn pandas
```

A screenshot of a Windows Command Prompt window. The title bar reads 'Command Prompt'. The command prompt shows the command 'C:\Users\dr.eet>pip install numpy scipy matplotlib ipython scikit-learn pandas' being entered. The cursor is at the end of the command line.

```
Command Prompt

C:\Users\dr.eet>pip install numpy scipy matplotlib ipython scikit-learn pandas_
```

Installing scikit-learn



```
Command Prompt

----- 56.0/56.0 kB 486.9 kB/s eta 0:00:00
Downloading pillow-10.3.0-cp312-cp312-win_amd64.whl (2.5 MB)
----- 2.5/2.5 MB 1.2 MB/s eta 0:00:00
Downloading pyparsing-3.1.2-py3-none-any.whl (103 kB)
----- 103.2/103.2 kB 458.2 kB/s eta 0:00:00
Downloading pytz-2024.1-py2.py3-none-any.whl (505 kB)
----- 505.5/505.5 kB 1.3 MB/s eta 0:00:00
Downloading threadpoolctl-3.4.0-py3-none-any.whl (17 kB)
Downloading tzdata-2024.1-py2.py3-none-any.whl (345 kB)
----- 345.4/345.4 kB 1.3 MB/s eta 0:00:00
Installing collected packages: pytz, tzdata, threadpoolctl, pyparsing, pillow, num
py, kiwisolver, joblib, fonttools, cyclер, scipy, pandas, contourpy, scikit-learn,
matplotlib
Successfully installed contourpy-1.2.1 cyclер-0.12.1 fonttools-4.51.0 joblib-1.4.0
kiwisolver-1.4.5 matplotlib-3.8.4 numpy-1.26.4 pandas-2.2.2 pillow-10.3.0 pyparsi
ng-3.1.2 pytz-2024.1 scikit-learn-1.4.2 scipy-1.13.0 threadpoolctl-3.4.0 tzdata-20
24.1

C:\Users\dr.eet>
```

NumPy

- ❖ NumPy is one of the fundamental packages for scientific computing in Python.
- ❖ It contains functionality for multidimensional arrays, high-level mathematical functions such as linear algebra operations and the Fourier transform, and pseudorandom number generators.
- ❖ In scikit-learn, the NumPy array is the fundamental data structure.
- ❖ Scikit-learn takes in data in the form of NumPy arrays and any data will convert to a NumPy array.
- ❖ The core functionality of NumPy is the ndarray class, a multidimensional (n-dimensional) array.

Essential Libraries and Tools



NumPy

A screenshot of a Jupyter Notebook interface in a web browser. The browser tabs show "Home" and "ch2". The address bar displays "localhost:8888/notebooks/ML%20Test/ch2.ipynb". The Jupyter header shows the "jupyter" logo, the notebook name "ch2", and "Last Checkpoint: 2 days ago". The menu bar includes "File", "Edit", "View", "Run", "Kernel", "Settings", and "Help". The toolbar contains icons for saving, adding, deleting, and running code. The notebook content shows a code cell with the following Python code:

```
[1]: import numpy as np
x=np.array([[1,2,3],[4,5,6]])
print("x:\n{}".format(x))
```

The output of the code cell is displayed below the code:

```
x:
[[1 2 3]
 [4 5 6]]
```

Below the output, there is an empty code cell with the prompt "[]:".

SciPy

- ❖ SciPy is a collection of functions for scientific computing in Python.
- ❖ It provides advanced linear algebra routines, mathematical function optimization, signal processing, special mathematical functions, and statistical distributions.
- ❖ Scikit-learn draws from SciPy's collection of functions for implementing its algorithms.
- ❖ The most important part of SciPy is **scipy.sparse**: provides sparse matrices, which are another representation that is used for data in scikit-learn.
- ❖ **Sparse matrices are used to store a 2D array that contains mostly zeros.**

SciPy

#Create a 2D NumPy array with a **diagonal of ones**, and zeros everywhere else

```
[2]: from scipy import sparse
     eye=np.eye(4)
     print("NumPy array:\n{}".format(eye))
```

```
NumPy array:
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]]
```

The `eye()` function is used to create a 2-D array with ones on the diagonal and zeros elsewhere. The `eye()` function is commonly used in linear algebra and matrix operations. It is useful for generating matrices to transform, rotate, or scale vectors. It can also be used in scientific computing for solving differential equations, optimization, and signal processing.

SciPy

```
#Convert the NumPy array to a SciPy sparse matrix in CSR format  
#Only the nonzero entries are stored
```

```
[3]: sparse_matrix=sparse.csr_matrix(eye)  
print("\nSciPy sparse CSR matrix:\n{}".format(sparse_matrix))
```

SciPy sparse CSR matrix:

(0, 0)	1.0
(1, 1)	1.0
(2, 2)	1.0
(3, 3)	1.0

Sparse data is data that mostly unused elements that don't carry any information.

Sparse Data is a data set where most of the item values are zero.

Dense Array is the opposite of a sparse array: most of the values are not zero.

Two types of sparse metrics are

CSC – Compressed Sparse Column. For efficient arithmetic, fast column slicing.

CSR - Compressed Sparse Row. For fast row slicing, faster matrix vector products

<https://docs.scipy.org/doc/scipy/reference/sparse.html>

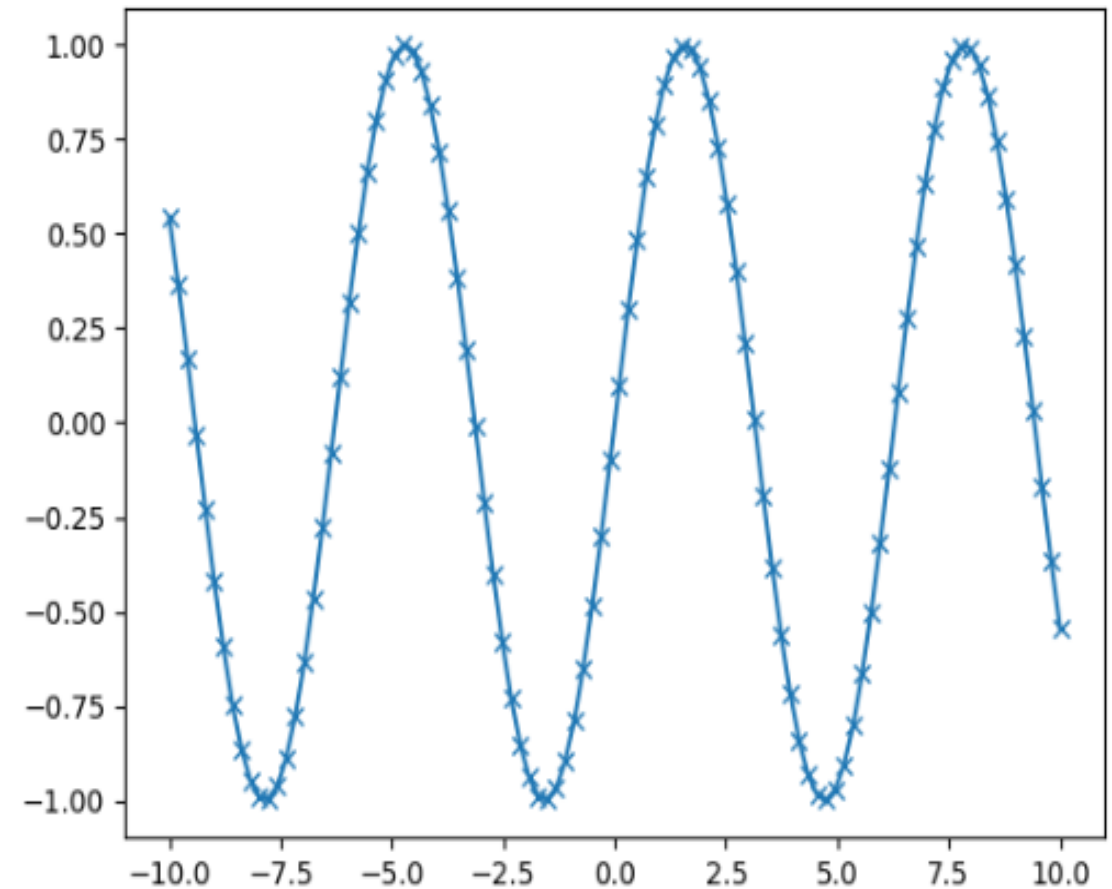
Essential Libraries and Tools

matplotlib

- ❖ matplotlib is the primary scientific plotting library in Python.
- ❖ It provides functions for making visualizations such as line charts, histograms, scatter plots, and so on.

```
[10]: import matplotlib.pyplot as plt
      x=np.linspace(-10,10,100)
      y=np.sin(x)
      plt.plot(x,y,marker="x")
```

[<matplotlib.lines.Line2D at 0x1ab76771d30>]



pandas

- ❖ pandas is a Python library for data wrangling and analysis.
- ❖ It is built around a data structure called DataFrame that is modeled after the R DataFrame.
- ❖ pandas DataFrame is a table, similar to an Excel spreadsheet.
- ❖ pandas provides a great range of methods to modify and operate on this table, it allows SQL-like queries and joins of tables.
- ❖ In contrast to NumPy, which requires that all entries in an array be of the same type, pandas allows each column to have a separate type (for example, integers, dates, floating-point numbers, and strings).
- ❖ pandas has ability to ingest from a great variety of file formats and databases, like SQL, Excel files and comma-separated vales (CSV) files.

pandas

```
[11]: import pandas as pd
data={'Name':["John","Anna","Peter","Linda"],
      'Location':["New York","Paris","Berlin","London"],
      'Age': [24,13,53,33]}
data_pandas=pd.DataFrame(data)
display(data_pandas)
```

	Name	Location	Age
0	John	New York	24
1	Anna	Paris	13
2	Peter	Berlin	53
3	Linda	London	33

pandas

```
[12]: display(data_pandas[data_pandas.Age>30])
```

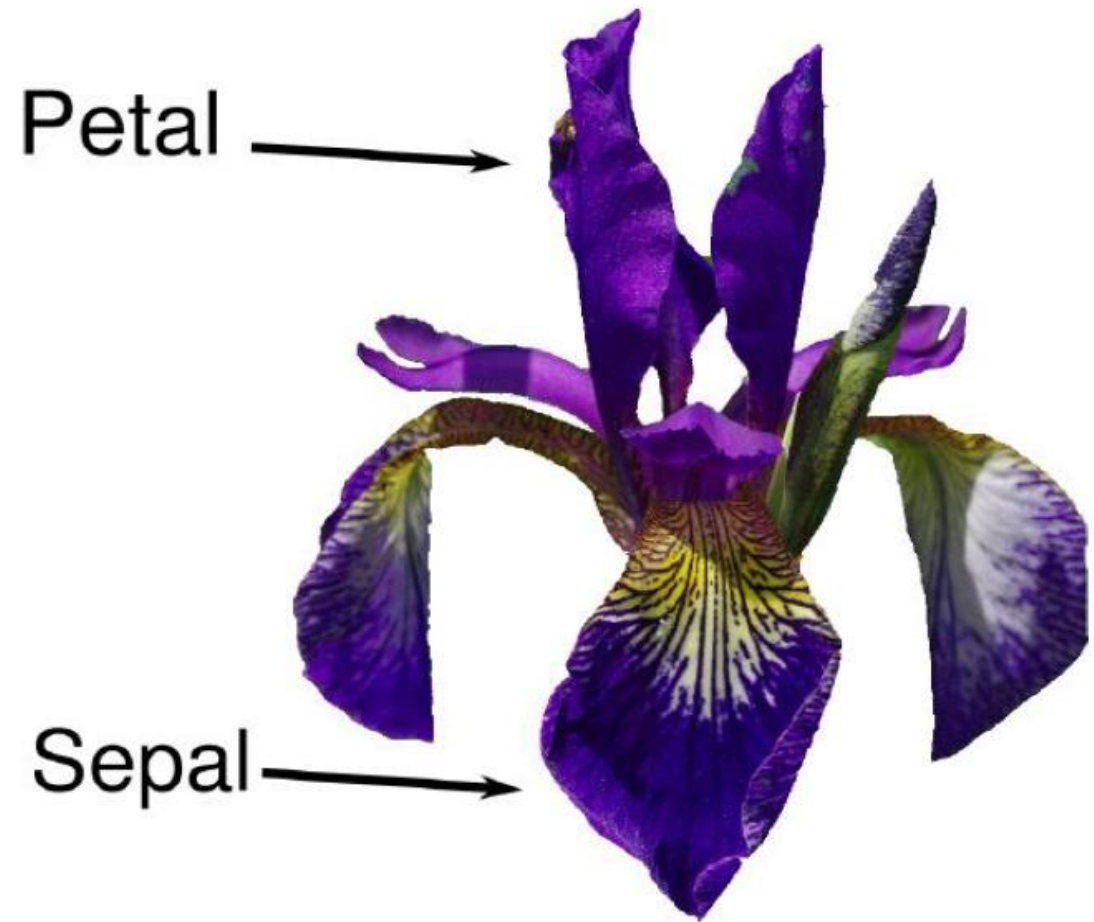
	Name	Location	Age
2	Peter	Berlin	53
3	Linda	London	33

Example: Classifying Iris Species

Create a simple **supervised** machine learning model for predicting Iris species.

Based on the width and length of **Petals** and **Sepals**, the model will predict the species of Iris: **Setosa**, **Versicolor**, or **Virginica**.

Iris dataset is a classical dataset in machine learning and statistics. It is included in scikit-learn in the datasets module.



scikit-learn comes with a few small standard datasets that do not require to download from external website: Iris, Diabetes, Digits, Linnerud, Wine, Breast Cancer

```
[15]: from sklearn.datasets import load_iris
iris_dataset=load_iris()
print("Keys of iris_dataset: \n{}".format(iris_dataset.keys()))

Keys of iris_dataset:
dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])
```

```
[16]: print(iris_dataset['DESCR'][:193]+"\\n...")

.. _iris_dataset:

Iris plants dataset
-----

**Data Set Characteristics:**

: Number of Instances: 150 (50 in each of three classes)
: Number of Attributes: 4 numeric, predictive
...
```

```
[22]: print("Target names:{}".format(iris_dataset['target_names']))
```

```
Target names:['setosa' 'versicolor' 'virginica']
```

```
[26]: print("Feature names:\n{}".format(iris_dataset['feature_names']))
```

```
Feature names:
```

```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

```
[27]: print("Type of data:{}".format(type(iris_dataset['data'])))
```

```
Type of data:<class 'numpy.ndarray'>
```

```
[28]: print("Shape of data:{}".format(iris_dataset['data'].shape))
```

```
Shape of data:(150, 4)
```

```
[29]: print("First five columns of data:\n{}".format(iris_dataset['data'][:5]))
```

```
First five columns of data:
```

```
[[5.1 3.5 1.4 0.2]
```

```
 [4.9 3.  1.4 0.2]
```

```
 [4.7 3.2 1.3 0.2]
```

```
 [4.6 3.1 1.5 0.2]
```

```
 [5.  3.6 1.4 0.2]]
```

```
[30]: print("Target:\n{}".format(iris_dataset['target']))
```

Target:

[illegible]

0 – setosa

1 – versicolor

2 - virginica

Measuring Success: Training and Testing Data

To assess the model's performance, splitting the labeled data into two parts: **training data** and **test data**. Use a test set containing 25% of the data is a good rule of thumb.

In scikit-learn, **data** is usually denoted with a **capital X**, while **labels** are denoted by a **lowercase y**.

This is inspired by the standard formulation $f(x)=y$ in mathematics, where x is the input to a function and y is the output.

Capital X: the data is a two-dimensional array (matrix)

Lowercase y: the target is a one-dimensional array (vector)

Before making the split, the **train_test_split** function shuffles the dataset using a pseudorandom number generator.

```
[33]: from sklearn.model_selection import train_test_split
      X_train, X_test, y_train, y_test = train_test_split(iris_dataset['data'], iris_dataset['target'], random_state=0)
```

The output of the train_test_split function is X_train, X_test, y_train, and y_test, which all are NumPy arrays.

X_train contains 75% of the rows of the dataset

X_test contains the remaining 25%

```
[34]: print("X_train shape: {}".format(X_train.shape))  
      print("y_train shape: {}".format(y_train.shape))
```

```
X_train shape: (112, 4)  
y_train shape: (112,)
```

```
[35]: print("X_test shape: {}".format(X_test.shape))  
      print("y_test shape: {}".format(y_test.shape))
```

```
X_test shape: (38, 4)  
y_test shape: (38,)
```

Building First Model: K-Nearest Neighbors

- ❖ Using k-nearest neighbors classifier to make a prediction for a new data point, the algorithm finds the point in the training set that is closet to the new point. Then it assigns the label of this training point to the new data point.
- ❖ The K in k-nearest neighbors signifies that instead of using only the closest neighbor to the new data point, consider any fixed number k of neighbors in the training.
- ❖ All machine learning models in scikit-learn are implemented in their own classes, which are called **Estimator classes**.

Building First Model: K-Nearest Neighbors

- ❖ The k-nearest neighbors classification algorithm is implemented in the KNeighborsClassifier class in the neighbors module.
- ❖ Need to instantiate the class into an object before using the model. This will set any parameters of the model. The most **important parameter** of KNeighborsClassifier is the **number of neighbors**.

```
[2]: from sklearn.neighbors import KNeighborsClassifier  
     knn=KNeighborsClassifier(n_neighbors=1)
```

- ❖ The knn object encapsulates the algorithm that will be used to build the model from the training data, as well the algorithm to make predictions on new data points.
- ❖ It will also hold the information that the algorithm has extracted from the training data. In the case of KNeighborsClassifier, it will just store the training set.

Building First Model: K-Nearest Neighbors

- ❖ To build the model on the training set, call the fit method of the knn object, which takes as arguments the NumPy array `X_train` containing the training data and the NumPy array `y_train` of the corresponding training labels.

```
[7]: knn.fit(X_train, y_train)
```

```
[7]: KNeighborsClassifier
      KNeighborsClassifier(n_neighbors=1)
```

- ❖ The fit method returns the knn object itself, so we get a string representation of classifier.

Making Predictions

- ❖ Make predictions using model on new data for which we might not know the correct labels.
 - ❖ Imagine an iris in the wild with a sepal length of 5 cm, a sepal width of 2.9 cm, a petal length of 1 cm, and a petal width of 0.2 cm.
 - ❖ What species of iris would this be?
-
- ❖ Put new data into NumPy array by calculating the shape.

```
[10]: X_new=np.array([[5, 2.9, 1, 0.2]])  
      print("X_new.shape: {}".format(X_new.shape))  
  
      X_new.shape: (1, 4)
```

Making Predictions

- ❖ We made the measurements of single flower into a row in a two-dimensional NumPy array, as scikit-learn always expects two-dimensional arrays for the data.
- ❖ To make a prediction, call the predict method of the knn object:

```
[11]: prediction=knn.predict(X_new)
      print("Prediction: {}".format(prediction))
      print("Predicted target name: {}".format(iris_dataset['target_names'][prediction]))

Prediction: [0]
Predicted target name: ['setosa']
```

- ❖ The KNN model predicts that new iris belongs to the **class 0**, meaning its species is **setosa**.

Evaluating the Model

- ❖ Test data was not used to build the model, but we do know what the correct species is for each iris in the test set. Therefore, we can make a prediction for each iris in the test data and **compare it against its label** (the known species). We can measure how well the model works by **computing the accuracy**, which is the fraction of flowers for which the right species was predicted:

```
[12]: y_pred=knn.predict(X_test)
      print("Test set predictions:\n {}".format(y_pred))
```

```
Test set predictions:
```

```
[2 1 0 2 0 2 0 1 1 1 2 1 1 1 1 0 1 1 0 0 2 1 0 0 2 0 0 1 1 0 2 1 0 2 2 1 0
2]
```

Evaluating the Model

- ❖ Test set accuracy is about 0.97, which means we made the right prediction for 97% of the Iris in the test set.
- ❖ Under some mathematical assumptions, this means that we can expect our model to be correct 97% of the time for new Iris.

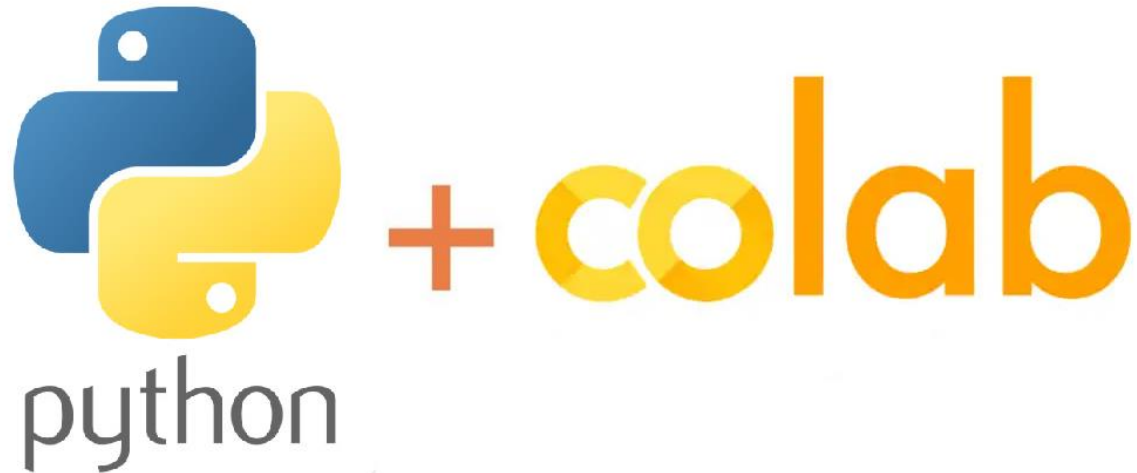
```
[15]: print("Test set score: {:.2f}".format(np.mean(y_pred==y_test)))
```

```
Test set score: 0.97
```

```
[17]: print("Test set score: {:.2f}".format(knn.score(X_test,y_test)))
```

```
Test set score: 0.97
```

Google Colab



<https://colab.google/>

ch2

Ch1.ipynb - Colab

colab.research.google.com/drive/1738_UgmmASUbbcqhtciAYE5GVkBI_vvz#scrollTo=LSTHbsQDz7Py

DrEET Copilot Gemini Google Cloud Skills... Artificial Intelligence Moodle RH124 - pr01s02 Hybrid Cloud Devel... Cmp Sc 4760 -- Op... Servlet Tutorial | Stu... All Bookmarks

Ch1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

RAM Disk

+ Code + Text

0s

[1] import numpy as np
x=np.array([[1,2,3],[4,5,6]])
print("x:\n{}".format(x))

x:
[[1 2 3]
 [4 5 6]]

0s

[2] from scipy import sparse
eye=np.eye(4)
print("NumPy array:\n{}".format(eye))

NumPy array:
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

0s

sparse_matrix=sparse.csr_matrix(eye)
print("\nSciPy sparse CSR matrix:\n{}".format(sparse_matrix))

SciPy sparse CSR matrix:

completed at 6:01 PM

Supervised Learning



Dr. Ei Ei Thu

Associate Professor

Faculty of Computer Science

Outlines



1

Supervised Machine Learning Problems

2

**Generalization, Overfitting and Underfitting
Supervised Machine Learning Algorithms**

3

**K-Nearest Neighbors Classifier
Naïve Bayes Classifier**

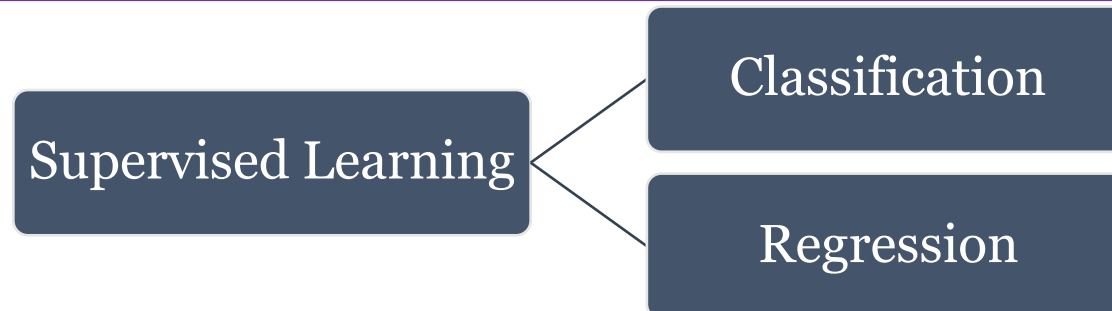


Supervised Learning

Supervised machine learning is a type of machine learning where the model is **trained on a labeled dataset**. In this approach, each training example is a **pair** consisting of an **input object and a desired output value**. The aim of supervised learning is **build a model** to learn a **mapping from input to outputs** that can be used **to predict** the **output** for **new unseen inputs**.

Types of Supervised Machine Learning Problems

There are two major types of supervised machine learning problems, called **classification** and **regression**.



Classification

- ❖ The goal of **classification** is to predict a class label, which is a choice from a predefined list of possibilities.
- ❖ Classification is separated into **binary classification**, which is the special case of distinguishing between exactly **two classes**, and **multiclass classification**, which is classification between **more than two classes**.
- ❖ Example: Classifying emails as either spam or not spam is an example of a **binary classification** problem. Iris flower classification is an example of multiclass classification.

- ❖ The goal of regression is to predict a **continuous number**, or a floating-point number in programming terms.
- ❖ Regression tasks is kind of continuity in the output. If there is **continuity** between possible **outcomes**, then the problem is a regression problem.
- ❖ Example: Predicting a person's annual income from their education, their age, and where they live is an example of a regression task. When predicting income, the predicted value is an amount, and can be any number in a given range. Another example of a regression task is predicting the yield of a corn farm given attributes such as previous yields, weather, and number of employees working on the farm. The yield again can be an arbitrary number.

Generalization, Overfitting, and Underfitting



- ❖ In supervised learning, build a model on the training data to make accurate predictions on new, unseen data.
- ❖ If a model is able to make **accurate predictions on unseen data**, it is able to **generalize** from the training set to the test set.
- ❖ Build a model that is too complex for the amount of information is called **overfitting**. Overfitting occurs when a model **fit too closely to the training set** and obtain a model that works well on the training set but is **not** able to **generalize** to new data.
- ❖ If a model is too simple and it might not be able to capture all the aspects of data variability, and it will do badly even on the training set is called **underfitting**.

Generalization, Overfitting, and Underfitting



- ❖ **Generalized Model:** A model that balances bias and variance.
- ❖ **Overfitted Model:** A model that is too complex and has **high variance**.
- ❖ **Underfitted Model:** A model that is too simple and has **high bias**.
- ❖ **Sweet Spot:** It refers to the optimal balance between bias and variance in a model, leading to the best possible performance on un seen data. The sweet spot is where the total error (sum of bias and variance) is minimized.

Bias-Variance Tradeoff



- ❖ **Bias:** Bias is the error introduced by approximating a real-world problem, which may be complex, by a simplified model. High bias can cause the model to miss relevant relations between features and target outputs (**underfitting**).
- ❖ **Variance:** Variance is the error introduced by the model's sensitivity to small fluctuations in the training set. High variance can cause the model to model the noise in the training data rather than the intended outputs (**overfitting**).

Trade-off between overfitting and underfitting

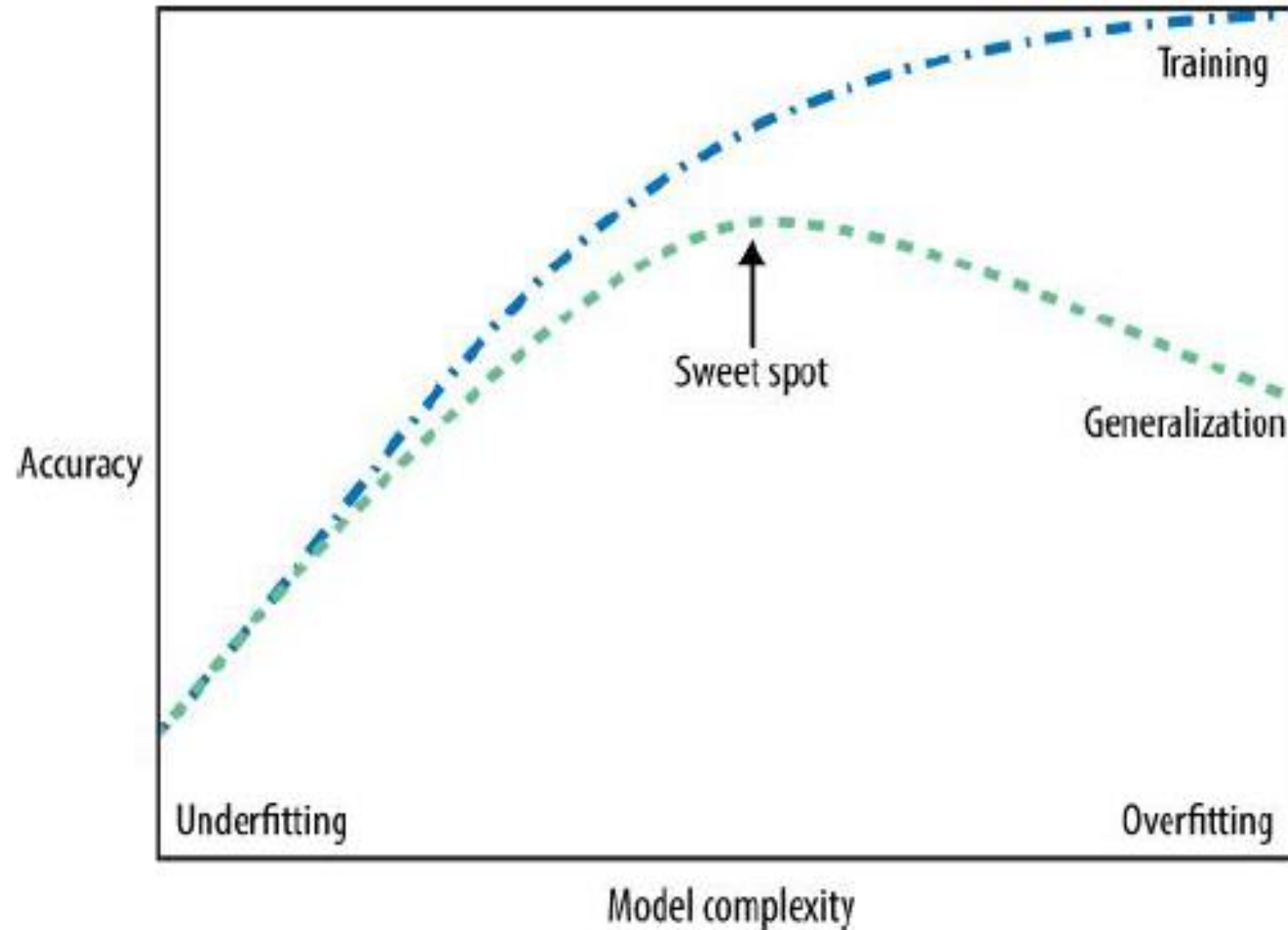


Figure 2-1. Trade-off of model complexity against training and test accuracy

Example Scenario: Predict Customer will buy a boat



Table 2-1. Example data about customers

Age	Number of cars owned	Owns house	Number of children	Marital status	Owns a dog	Bought a boat
66	1	yes	2	widowed	no	yes
52	2	yes	3	married	no	yes
22	0	no	0	married	yes	no
25	1	no	1	single	no	no
44	0	no	2	divorced	yes	no
39	1	yes	2	married	yes	no
26	1	no	2	single	no	no
40	3	yes	1	married	yes	no
53	2	yes	2	divorced	no	yes
64	2	yes	3	divorced	no	no
58	2	yes	2	married	yes	yes
33	1	no	1	single	no	no

Example Scenario: Predict Customer will buy a boat

- ❖ Table 2.1 given records of previous boat buyers and customers to predict where a new customer will buy a boat or not.
- ❖ The goal is to send out promotional emails to people who are likely to actually make a purchase.

“Everybody who owns a house buys a boat.”

“If the customer is older than, 45, and has less than 3 children or is not divorced, then they want to buy a boat.”

“People older than 50 want to buy a boat.”

Age	No. cars	Owens house	No. child	Marital	Owens dog	Bought a boat
66	1	yes	2	widowed	no	yes
52	2	yes	3	married	no	yes
53	2	yes	2	divorced	no	yes
58	2	yes	2	married	yes	yes

Age	No. cars	Owens house	No. child	Marital	Owens dog	Bought a boat
22	0	no	0	married	yes	no
25	1	no	1	single	no	no
44	0	no	2	divorced	yes	no
39	1	yes	2	married	yes	no
26	1	no	2	single	no	no
40	3	yes	1	married	yes	no
64	2	yes	3	divorced	no	no
33	1	no	1	single	no	no

Model 1: “Everybody who owns a house buys a boat.”

House owners who bought a boat: 4 out of 7

House owners who did not buy a boat: 3 out of 7

Owns house	Bought a boat
yes	yes
yes	yes
yes	yes
yes	yes
yes	no
yes	no
yes	no

It has **high bias**, making a broad generalization based on a single feature.

It ignores other important variables in the data.

It oversimplifies the prediction task, leading to inaccurate predictions for new cases.

Therefore, Model-1 is an underfitted model.

Model 2: “If the customer is older than, 45, and has less than 3 children or is not divorced, then they want to buy a boat.”

Age	No. child	Marital	Bought a boat
66	2	widowed	yes
52	3	married	yes
53	2	divorced	yes
58	2	married	yes
64	3	divorced	no

It incorporates multiple specific conditions which may be based on limited data points.

It captures patterns that are specific to the training data and may not generalize well to other datasets.

The inclusion of the condition "or is not divorced" hinges on a single customer's data, indicating a reliance on specific details rather than a general pattern.

According to the author's statement, Model-2 is **more likely** to be overfitted model.

Model 3: “People older than 50 want to buy a boat.”

Age	Bought a boat
66	yes
52	yes
53	yes
58	yes
64	no

If the dataset shows a clear trend that older people are more likely to buy boats, this model captures this general trend effectively without overfitting to specific noise.

If the dataset shows a strong trend that most people over 50 buy boats, this model could be considered generalized because it captures a broad, general pattern.

The characterization of this model largely depends on the dataset and context provided by the author.

According to the author’s statement, Model-2 is **more likely** to be generalized model.

Relation of Model Complexity to Dataset Size

- ❖ The model complexity is intimately tied to the variation of inputs contained in training dataset: the larger variety of data points contain in dataset, the more complex a model.
- ❖ Collecting more data points will yield more variety, so larger datasets allow building more complex models.
- ❖ However, simply duplicating the same data points or collecting very similar data will not help.

Supervised Learning Algorithms



Classification

- ❖ **K-Nearest Neighbors**
- ❖ **Naïve Bayes**
- ❖ Logistic Regression
- ❖ Support Vector Machine (SVM)
- ❖ Decision Tree
- ❖ Random Forest
- ❖ Neural Networks

Regression

- ❖ Linear Regression
- ❖ Support Vector Machine (SVM)
- ❖ Decision Tree
- ❖ Random Forest
- ❖ Neural Networks

K-Nearest Neighbors (K-NN)



Type: Classification and Regression

Description: Classifies a data point based on how its neighbors are classified, with the 'K' parameter defining the number of neighbors to consider.

Use Cases: Recommendation Systems, Pattern Recognition

Naïve Bayes



Type: Classification

Description: Based on Bayes' theorem, assumes that the features are independent given the class label.

Use Cases: Text Classification, Sentiment Analysis

Logistic Regression



Type: Classification

Description: Models the probability that an instance belongs to a particular class using a logistic function.

Use Cases: Binary classification problems like spam detection, disease diagnosis

Support Vector Machine (SVM)



Type: Classification and Regression

Description: Finds the hyperplane that best separates the classes in the feature space, maximizing the margin between different classes.

Use Cases: Image Classification, Text Categorization

Decision Tree



Type: Classification and Regression

Description: Uses a tree-like model of decisions and their possible consequences, splitting the data into subsets based on feature values.

Use Cases: Customer Segmentation, Loan Approval

Random Forests



Type: Classification and Regression

Description: An ensemble of decision trees, where each tree is trained on random subset of the data and the final prediction is made by averaging the predictions of all trees or by majority voting.

Use Cases: Customer Segmentation, Loan Approval

Type: Classification and Regression

Description: Composed of layers of interconnected nodes, where each node applies a nonlinear transformation to the input. Deep neural networks have many layers.

Use Cases: Image and Speech Recognition, Natural Language Processing

Workflow of Supervised Learning



- 1. Data Collection:** Gather the data that includes input features and corresponding labels.
- 2. Data Preprocessing:** Clean the data, handle missing values, **encode categorical variables**, and normalize/standardize the data if necessary.
- 3. Train-Test Split:** Split the data into training and testing sets to evaluate the model's performance on unseen data.
- 4. Model Selection:** Choose **an appropriate algorithm** based on the problem type and data characteristics.
- 5. Training:** Train the model on the training dataset.
- 6. Evaluation:** Evaluate the model using the testing dataset and relevant **metrics**.
- 7. Hyperparameter Tuning:** Adjust the hyperparameters of the model to improve performance.
- 8. Prediction:** Use the trained model to make predictions on new, unseen data.

Advantages of Supervised Learning



- ❖ **High Accuracy:** When enough labeled data is available, supervised learning models can achieve high accuracy.
- ❖ **Interpretability:** Many supervised learning algorithms provide models that are easy to interpret and understand.
- ❖ **Versatility:** Can be used for a wide range of tasks including classification, regression, and more.

Challenges Supervised Learning



- ❖ **Data Dependency:** Requires a large amount of labeled data, which can be expensive and time-consuming to collect.
- ❖ **Overfitting:** Risk of overfitting to the training data if the model is too complex, which can reduce its performance on unseen data.
- ❖ **Bias:** If the labeled data is not representative of the real-world scenario, the model can learn biased patterns.

K-Nearest Neighbors Classification

- ❖ The KNN algorithm compares a new data entry to the data in a given dataset based on its closeness or similarities in a given range (k) of neighbors, the algorithm assigns the new data to a class or category in the dataset (training data).

K-Nearest Neighbors Algorithm

Step 1: **Assign K value**

Step 2: **Calculate the distance** between the new data entry and all other existing data entries. Arrange them in ascending order.

Step 3: **Find the K nearest neighbors** to the new entry based on calculated distances.

Step 4: **Assign the new data entry** to the majority class in the nearest neighbors.

Distance Metrics used in K-NN Algorithm



- ❖ Euclidean Distance
- ❖ Manhattan Distance
- ❖ Minkowski Distance
- ❖ Hamming Distance
- ❖ Jaccard Distance

Euclidean Distance

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + \dots + (x_{2n} - x_{1n})^2}$$

x_2 = *New Data Entry*

x_1 = *Existing Data Entry*

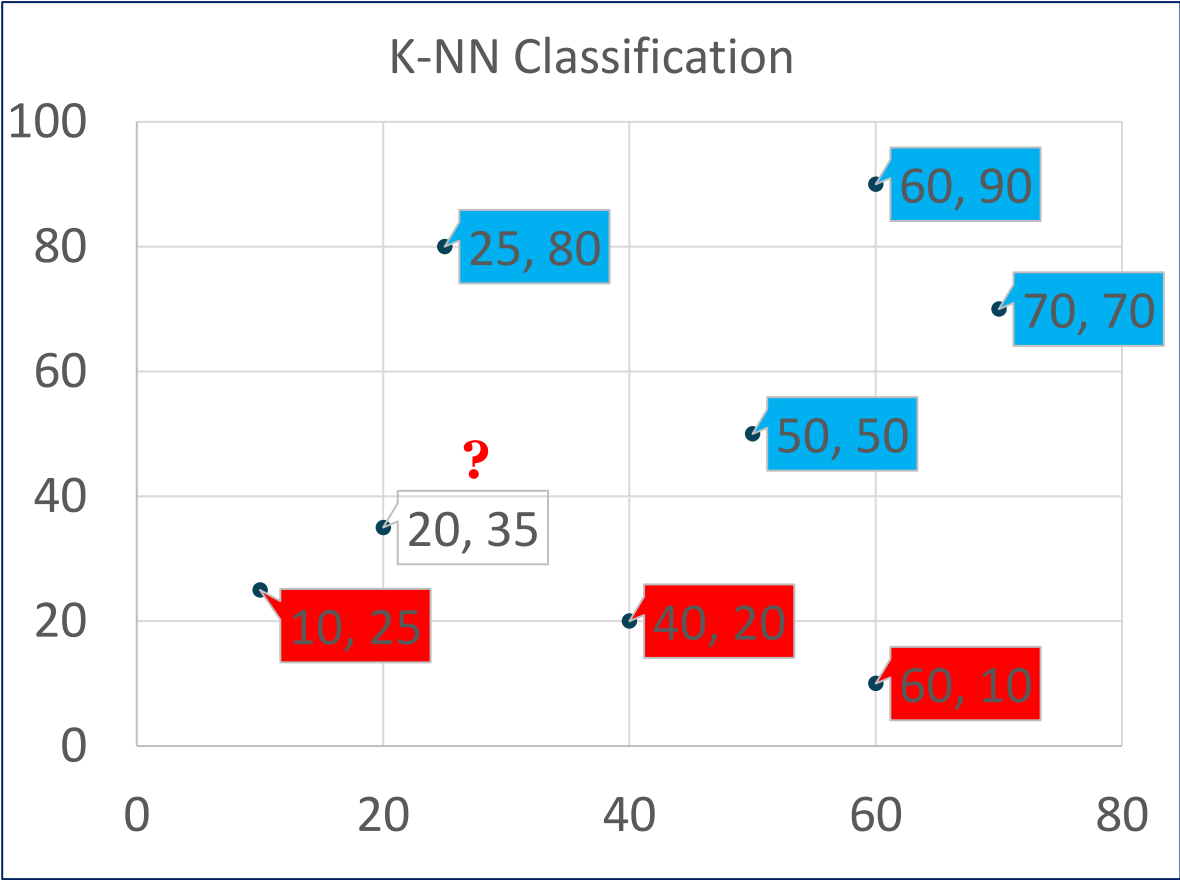
K-NN Classification Sample Calculation

Brightness	Saturation	Class
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue

New Data Entry

Brightness	Saturation	Class
20	35	?

K=3

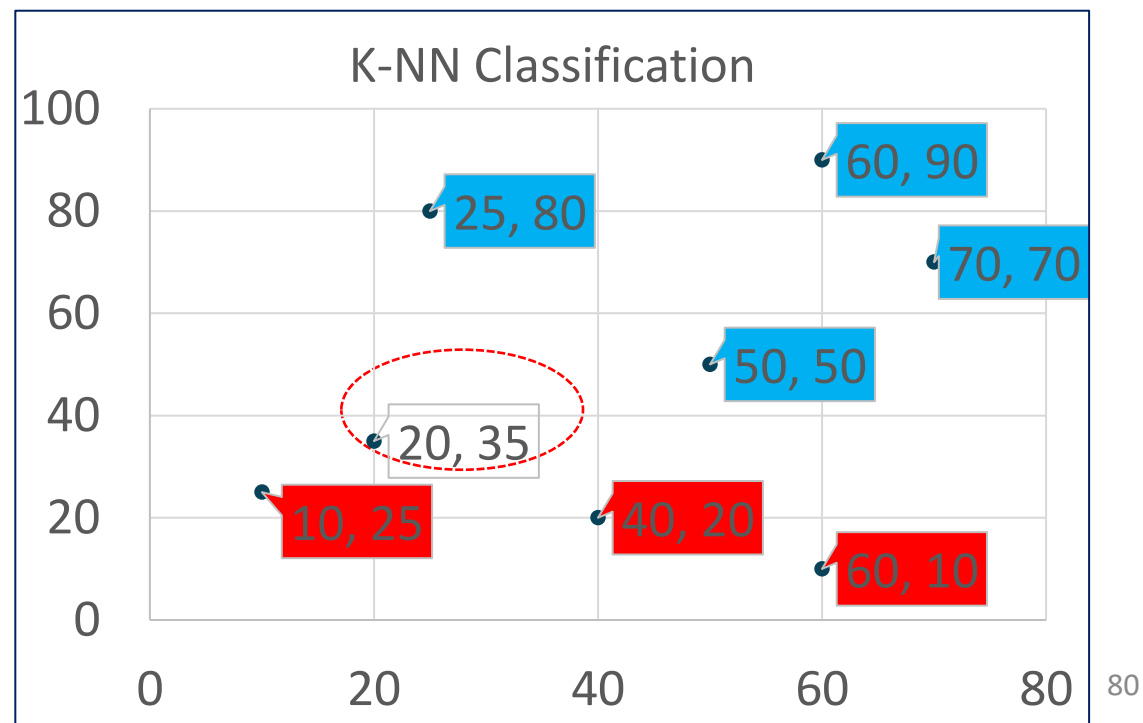


Existing Data (x1)			New Data (x2)			Distance
Brightness	Saturation	Class	Brightness	Saturation	Class	
40	20	Red	20	35	?	25.00
50	50	Blue	20	35		33.54
60	90	Blue	20	35		68.01
10	25	Red	20	35		14.14
70	70	Blue	20	35		61.03
60	10	Red	20	35		47.17
25	80	Blue	20	35		45.28

**Calculate
Euclidean Distance**

Arrange Ascending Order

Existing Data (x1)			Distance	K=3
Brightness	Saturation	Class		
10	25	Red	14.14	
40	20	Red	25.00	
50	50	Blue	33.54	
25	80	Blue	45.28	
60	10	Red	47.17	
70	70	Blue	61.03	
60	90	Blue	68.01	



```

import numpy as np
import pandas as pd

data_set=pd.read_csv('color.csv')

display(data_set)

X=data_set[['Brightness','Saturation']]

y=data_set[['Class']]

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test=train_test_split(X,y,random_state=0)

from sklearn.neighbors import KNeighborsClassifier
knn=KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train.values,y_train.values.ravel())

X_new=np.array([[20,35]])
prediction=knn.predict(X_new)
print(prediction)

from sklearn.metrics import accuracy_score

train_accuracy = accuracy_score(y_train, knn.predict(X_train))
print(train_accuracy)
print(knn.predict(X_train))

test_accuracy = accuracy_score(y_test, knn.predict(X_test))
print(test_accuracy)
print(knn.predict(X_test))

```

K-NN Sample Code for color classification

```

import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score
data_set=pd.read_csv('iris2.csv')
display(data_set)
print(data_set.keys())

X=data_set[['sepal_length','sepal_width','petal_length','petal_width']]
y=data_set[['species']]

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test=train_test_split(X,y,random_state=2)

from sklearn.neighbors import KNeighborsClassifier
knn=KNeighborsClassifier(n_neighbors=4)
knn.fit(X_train.values,y_train.values.ravel())

X_new=np.array([[7.1,3.0,5.9,2.1]])
prediction=knn.predict(X_new)
print(prediction)

train_accuracy = accuracy_score(y_train, knn.predict(X_train))
print(train_accuracy)
|
test_accuracy = accuracy_score(y_test, knn.predict(X_test))
print(test_accuracy)

```

K-NN Sample Code or IRIS flower Classification

- ❖ Naïve Bayes classifier tends to be faster in training phase. Naïve Bayes model often provides generalization performance. The reason that naïve Bayes models are so efficient is that they learn parameters by looking at each feature individually and collect simple per-class statistics from each feature. There are three kinds of naïve Bayes classifiers:
 - ❖ Gaussian Naïve Bayes
 - ❖ Bernoulli Naïve Bayes
 - ❖ Multinomial Naïve Bayes
- ❖ GaussianNB can be applied to any **continuous data**, while BernoulliNB assumes **binary data** and MultinomialNB assumes **count data**. MultinomialNB takes into account the average value of each feature for each class. GaussianNB stores the average value as well as the standard deviation of each feature for each class.

Naïve Bayes Classifier



- ❖ BernoulliNB and MultinomialNB are mostly used in text data classification.
- ❖ BernoulliNB classifier counts every feature of each class is not zero.

```
[1]: import numpy as np
X=np.array([[0,1,0,1],
           [1,0,1,1],
           [0,0,0,1],
           [1,0,1,0]])
y=np.array([0,1,0,1])
counts={}
for label in np.unique(y):
    counts[label]=X[y==label].sum(axis=0)
print("Feature counts:\n{}".format(counts))
```

Counting the nonzero
entries per class

```
Feature counts:
{0: array([0, 1, 0, 2]), 1: array([2, 0, 2, 1])}
```

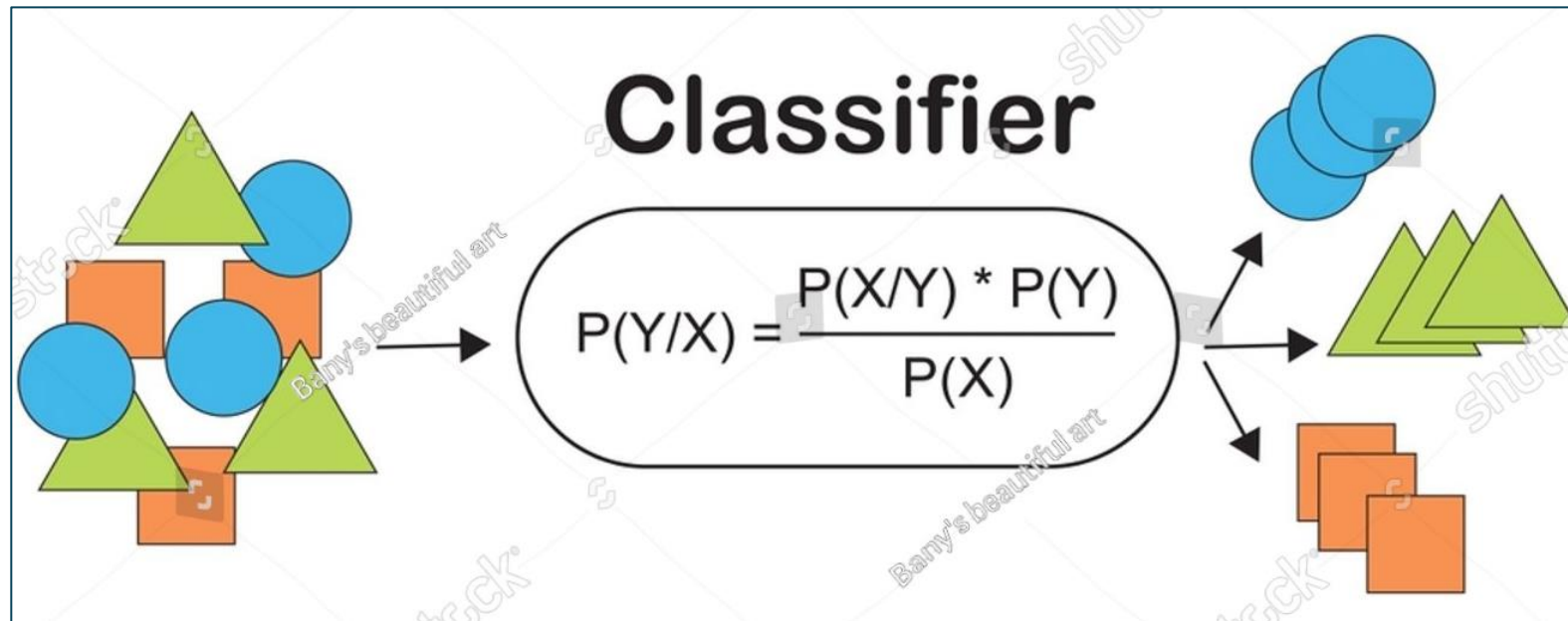
Naïve Bayes Steps

Step 1: Convert the dataset into a frequency table.

Step 2: Create Likelihood table by finding the probabilities.

Step 3: Use NB equation to calculate posterior probability.

The highest posterior probability is the outcome of prediction.



Bayes's Theorem

$$P(y|X) = \frac{P(X|y) * P(y)}{P(X)}$$



Thomas Bayes
1702 - 1761

y – Hypothesis

X – Evidence

P(y) – **Prior Probability**, Probability of occurrence of Hypothesis

P(X) – **Marginal Probability**, Probability of occurrence of evidence

P(X|y) – **Likelihood Probability**, Probability of occurrence of X given that y has already occurred.

P(y|X) – **Posterior Probability**, Probability of occurrence of y given that X has already occurred.

Naïve Bayes Classifier Sample Calculation

Consider a car stolen problem with attributes: Color, Type, Origin and the target, stolen can be either Yes or No.

Color	Type	Origin	Stolen?
Red	Sports	Domestic	Yes
Red	Sports	Domestic	No
Red	Sports	Domestic	Yes
Yellow	Sports	Domestic	No
Yellow	Sports	Imported	Yes
Yellow	SUV	Imported	No
Yellow	SUV	Imported	Yes
Yellow	SUV	Domestic	No
Red	SUV	Imported	No
Red	Sports	Imported	Yes

Classify a Red Domestic SUV is getting stolen or not.

$$P(y|X) = \frac{P(X|y) * P(y)}{P(X)}$$

y – class variable

X – Features

X = x1, x2, x3, ..., xn

Prior Probability

$P(\text{Yes}) = 5/10 = 1/2$
 $P(\text{No}) = 5/10 = 1/2$

Step 1: Convert the dataset into a **frequency** table.

Color	Stolen?	
	Yes	No
Red	3	2
Yellow	2	3

Type	Stolen?	
	Yes	No
Sports	4	1
SUV	1	3

Origin	Stolen?	
	Yes	No
Domestic	2	3
Imported	3	2

Step 2: **Likelihood** table by finding the probabilities.

Color	Stolen?	
	P(Yes)	P(No)
Red	3/5	2/5
Yellow	2/5	3/5

Type	Stolen?	
	P(Yes)	P(No)
Sports	4/5	1/5
SUV	1/5	3/5

Origin	Stolen?	
	P(Yes)	P(No)
Domestic	2/5	3/5
Imported	3/5	2/5

Step 3: Use NB equation to calculate posterior probability.

The highest posterior probability is the outcome of prediction.

$$P(y|X) = \frac{P(X|y) * P(y)}{P(X)}$$

Color	Type	Origin	Stolen
Red	SUV	Domestic	?

$$\begin{aligned}
 P(\text{Yes}|X) &= P(\text{Red}|\text{Yes}) * P(\text{SUV}|\text{Yes}) * P(\text{Domestic}|\text{Yes}) * P(\text{Yes}) \\
 &= 3/5 * 1/5 * 2/5 * 1/2 \\
 &= 0.024
 \end{aligned}$$

$$\begin{aligned}
 P(\text{No}|X) &= P(\text{Red}|\text{No}) * P(\text{SUV}|\text{No}) * P(\text{Domestic}|\text{No}) * P(\text{No}) \\
 &= 2/5 * 3/5 * 3/5 * 1/2 \\
 &= 0.072
 \end{aligned}$$

According to the outcome, Red Domestic SUV cannot getting stolen.

Gaussian Naïve Bayes Sample Code



```
import numpy as np
import pandas as pd

data_set=pd.read_csv('car.csv')
display(data_set)

from sklearn.preprocessing import LabelEncoder

df=pd.DataFrame(data_set)

le=LabelEncoder()
df['Color']=le.fit_transform(df['Color'])
df['Type']=le.fit_transform(df['Type'])
df['Origin']=le.fit_transform(df['Origin'])

df['Stolen']=le.fit_transform(df['Stolen'])
|
X=df.drop('Stolen',axis=1)
y=df['Stolen']
```

Gaussian Naïve Bayes Sample Code

Color	Type	Origin	Stolen?
Red	Sports	Domestic	Yes
Red	Sports	Domestic	No
Red	Sports	Domestic	Yes
Yellow	Sports	Domestic	No
Yellow	Sports	Imported	Yes
Yellow	SUV	Imported	No
Yellow	SUV	Imported	Yes
Yellow	SUV	Domestic	No
Red	SUV	Imported	No
Red	Sports	Imported	Yes

Label Encoding



Color	Type	Origin
0	1	0
0	1	0
0	1	0
1	1	0
1	1	1
1	0	1
1	0	1
1	0	0
0	0	1
0	1	1

Gaussian Naïve Bayes Sample Code

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test=train_test_split(X,y,random_state=0)

from sklearn.naive_bayes import GaussianNB
gnb=GaussianNB()
gnb.fit(X_train,y_train)

X_new=np.array([[0,0,0]])
prediction=gnb.predict(X_new)
print(prediction)
```

[0]

```
from sklearn.metrics import accuracy_score

print(gnb.predict(X_train))
print(y_train)

train_accuarcy=accuracy_score(y_train, gnb.predict(X_train))
print(train_accuarcy)

print(gnb.predict(X_test))
test_accuarcy=accuracy_score(y_test, gnb.predict(X_test))
print(test_accuarcy)
```

Unsupervised Learning



MACHINE
LEARNING

Dr. Ei Ei Thu

Associate Professor

Faculty of Computer Science

Outlines



1

Challenges of Unsupervised Learning

2

Preprocessing and Scaling
Applying Data Transformation

3

K-Means Clustering



Unsupervised Learning and Preprocessing



- ❖ Unsupervised learning subsumes all kinds of machine learning where there is **no known output**, no teacher to instruct the learning algorithm.
- ❖ In unsupervised learning, the learning algorithm is **just shown the input data** and asked to extract knowledge from this data.
- ❖ Two kinds of unsupervised learning: **Transformations of the dataset** and **clustering**.

Supervised Vs Unsupervised Learning, Explained

Supervised

X ₁	X ₂	X _p	Y

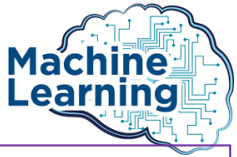
Target

Un-Supervised

X ₁	X ₂	X _p	Y

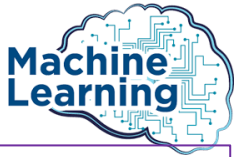
No
Target

Unsupervised Transformations



- ❖ Unsupervised transformations of a dataset are algorithms that **create a new representation of the data** which might be easier for humans or other machine learning algorithms to understand compared to the original representation of the data.
- ❖ A common application of unsupervised transformations is **dimensionality reduction**, which takes a high-dimensional representation of the data, consisting of many features, and finds a new way to represent this data that **summarizes the essential characteristics** with **fewer features**.
- ❖ A common application for dimensionality reduction is reduction to two dimensions for **visualization** purposes.
- ❖ Another application for unsupervised transformations is finding the parts or components that “make up” the data.

Example of Unsupervised Transformations



- ❖ **Topic extraction** on collections of text document, and to learn what topics appear in each document.
- ❖ Topic extraction can be useful for tracking the discussion of theme like elections, gun control, or pop stars on social media.

Clustering

- ❖ Partition data into **distinct groups of similar items**.
- ❖ Example: Uploading photos to a social media site. To organize pictures, the site might want to group together pictures that show the same person.

Challenges in Unsupervised Learning



- ❖ A major challenge in unsupervised learning is **evaluating** whether the algorithm learned something useful.
- ❖ Unsupervised learning algorithms are usually applied to data that does not contain any label information, so it doesn't know what the right output should be. It is very hard to say whether a model “did well”.
- ❖ Unsupervised algorithms are used often in an **exploratory** setting, when a data scientist wants to understand the data better, rather than as part of a larger automatic system. Another common application for unsupervised algorithms is as a **preprocessing step for supervised algorithms**.
- ❖ **Learning a new representation** of the data can sometimes **improve the accuracy of supervised algorithms**, or can lead to reduced memory and time consumption.

Data Types

I. Quantitative Data

❖ Numerical

- **Discrete:** counted data (no. of students)
- **Continuous:** measured data (price, size)

II. Qualitative Data

❖ Categorical

- **Nominal:** no inherent order or ranking (Colors, Gender)
- **Ordinal:** a specific order or ranking (Grades: A, B, Likert Scale)

Quantitative variables are variables that can be counted or measured.

Qualitative variables are variables that have distinct categories according to some characteristic or attribute.

Determine whether the data are **qualitative** or **quantitative**.

1. Color of football uniforms
2. Pizza sizes (s, m, l)
3. Cholesterol counts for individuals
4. Lightbulb wattage
5. Genders of movie stars
6. Ratings of teachers
7. Zip Code
8. Political Affiliation
9. Judging (1st, 2nd, 3rd)
10. Ranking of football players
11. No of children in family
12. Temperature

Determine whether the data are **discrete** or **continuous**.

1. Number of text messages send in a month
2. Systolic blood pressure readings
3. Heights of basketball players on a local high school team
4. Number of phone calls received during one day at a poison control center
5. Number of students in the mathematics classes during the fall semester at your school for a particular school year
6. Temperatures at a seashore resort
7. The highest wind speed of a hurricane
8. The weight of baggage on an airplane
9. The number of pages in a python book.
10. The amount of money a person spends per year for online purchases

Determine whether the data are **Nominal** or **Ordinal**.

1. The ratings of movies released this month
2. Colors of athletic shirts sold by Health Club
3. Telephone numbers
4. Ratings of fiction books: Excellent, good, fair, poor
5. Blood types: O, A, B, AB
6. Eye Color
7. Nationality
8. GPA Grade (A, B, C)
9. Major Field (CS, CT)
10. Religious Affiliation
11. Marital Status
12. Leap Year
13. Favorite Fruit
14. Educational Attainment
15. Type of pets

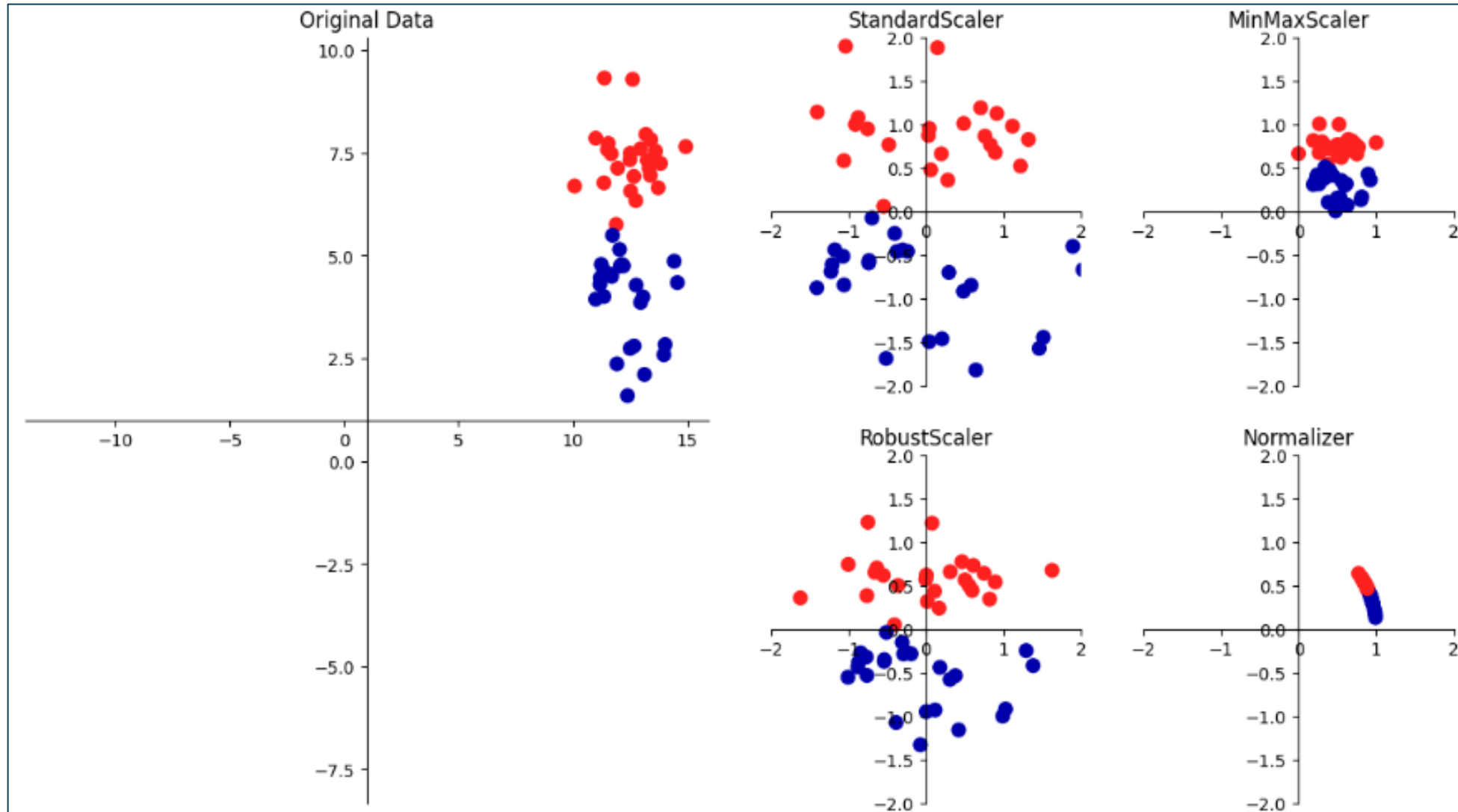
Mean, Median, Mode are measures of the central tendency. Measure of central tendency is the representation of various values of the given data set.

- ❖ Mean (μ) – The mean, or arithmetic average, is calculated by summing all the values in a dataset and dividing by the total number of values. It's **sensitive to outliers** and it is commonly used when the data is symmetrically distributed.
- ❖ Median (M) – The median is the middle value when the dataset is arranged in ascending or descending order. It is **robust to outliers** and is often used when the data is skewed.
- ❖ Mode (Z) – The mode is the value that occurs **most frequently** in the dataset. It can be applied to both numerical and categorical data.
- ❖ Variance (σ^2) – Variance is a measure of how data points **differ from the mean**.¹⁰⁴

Preprocessing and Scaling

A synthetic two-class classification dataset with **two features**. The first feature (the **x-axis** value) is between **10 and 15**. The second feature (the **y-axis** value) is between around **1 and 9**.

```
[2]: import mglearn
      mglearn.plots.plot_scaling()
```

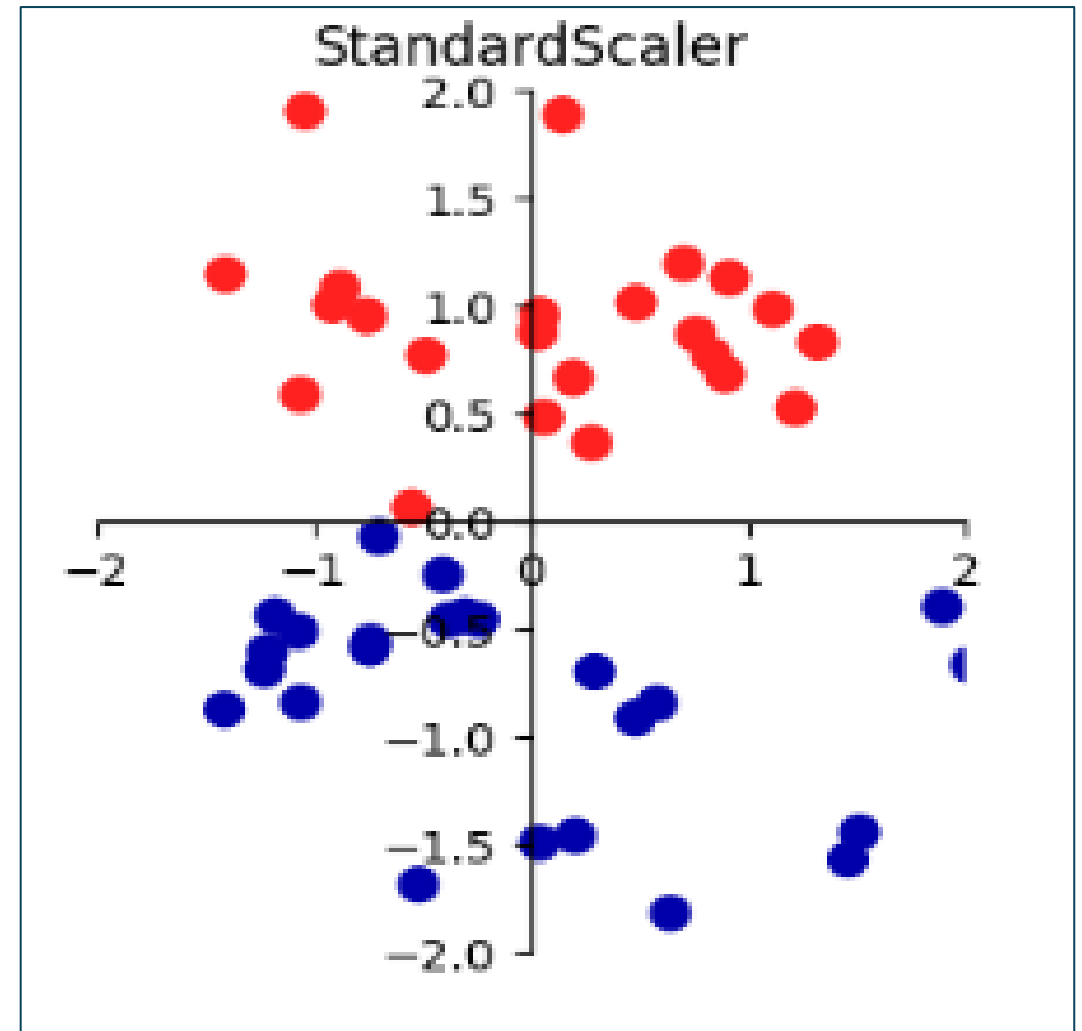
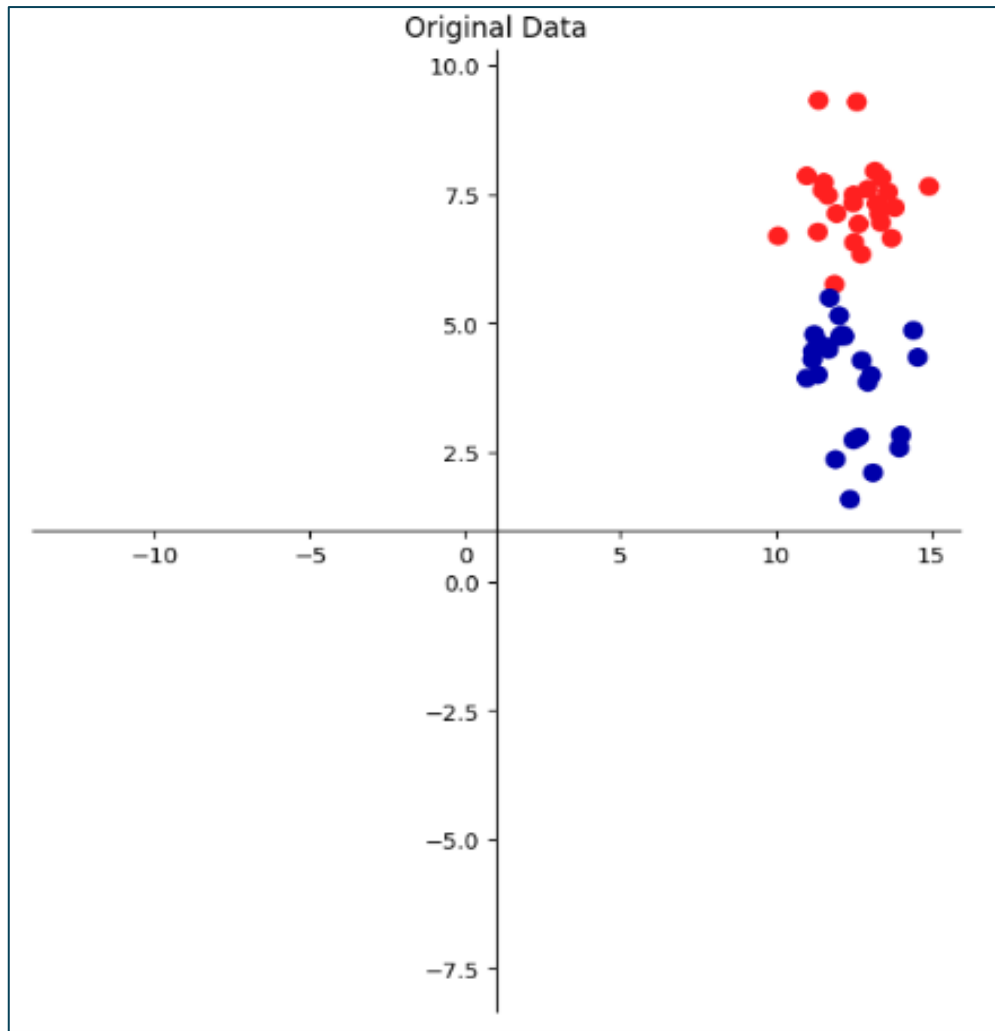


Scalars and Magnitude

- ❖ There are many quantities which require only one measurement to describe them. Such quantities are called **scalars**. Any quantity which can be represented as a number (positive or negative) is called scalar. This value is known as **magnitude**.
- ❖ There are four different ways to transform the data that yield more standard ranges:
 - ❖ StandardScaler
 - ❖ RobustScaler
 - ❖ MinMaxScaler
 - ❖ Normalizer

Standard Scaler

- ❖ The StandardScaler in scikit-learn ensures that for each feature the **mean** is **0** and the **variance** is **1**, bringing all features to the same magnitude.



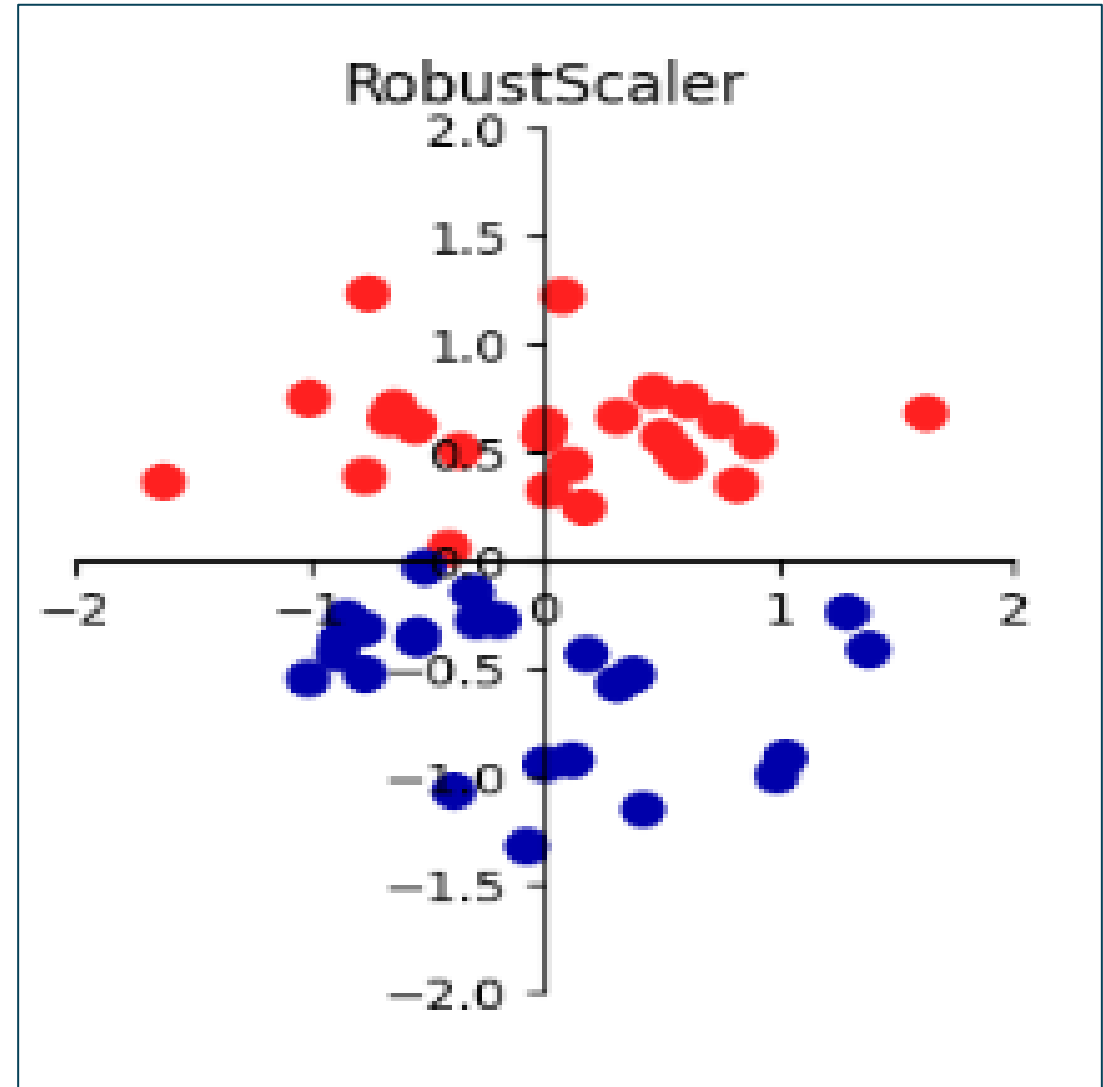
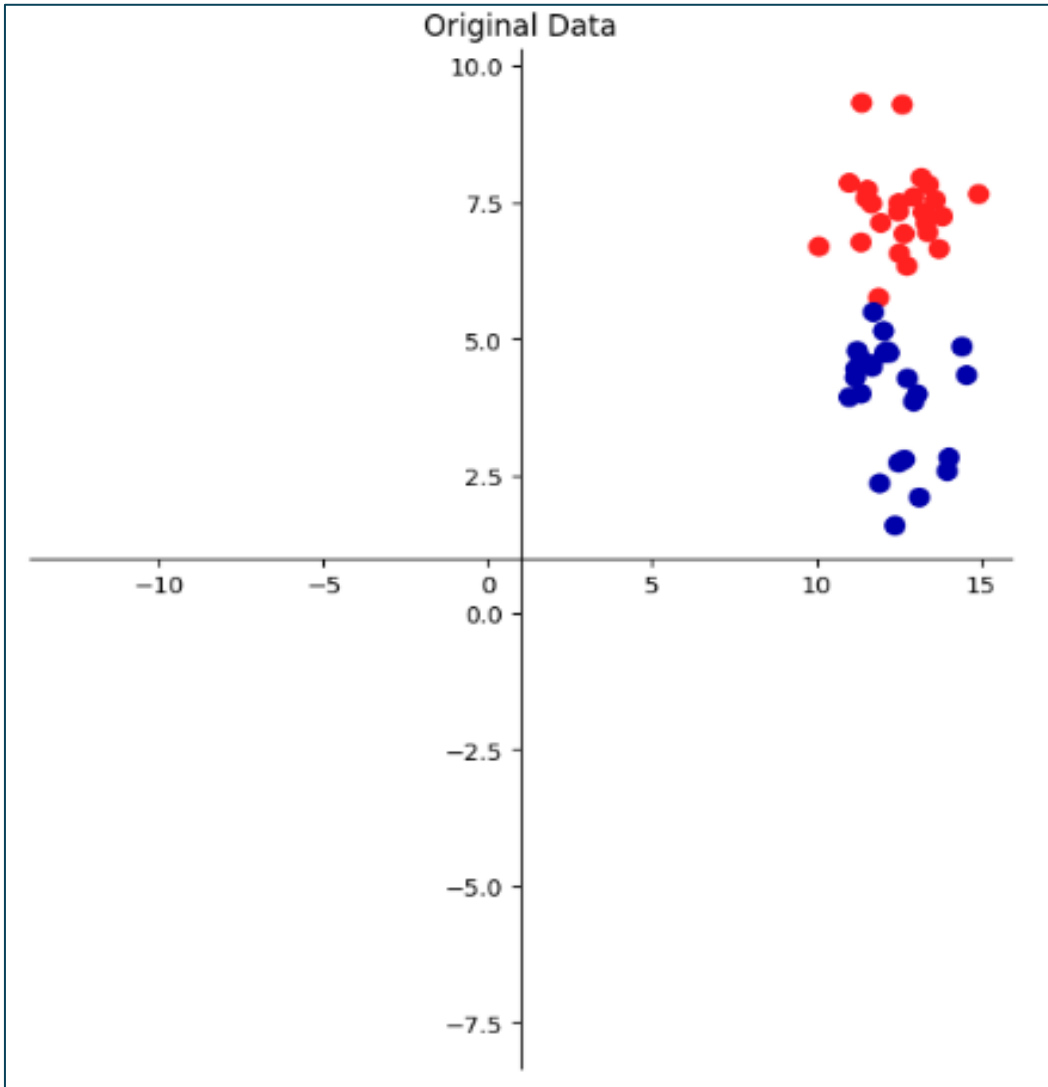
- ❖ The RobustScaler works similarly to the StandardScaler in that it ensures statistical properties for each feature that guarantee that they are on the same scale.
- ❖ The RobustScaler uses the **median and quartiles**, instead of mean and variance. This makes the RobustScaler ignore data points that are very different from the rest (like measurement errors). These **odd data points** are also called **outliers**, and can lead to trouble for other scaling techniques.

- ❖ Quartiles are values that divide data into quarters. The four quarters that divide a data set into quartiles are:

- ❖ The lowest 25% of numbers.
- ❖ The next lowest 25% of numbers.

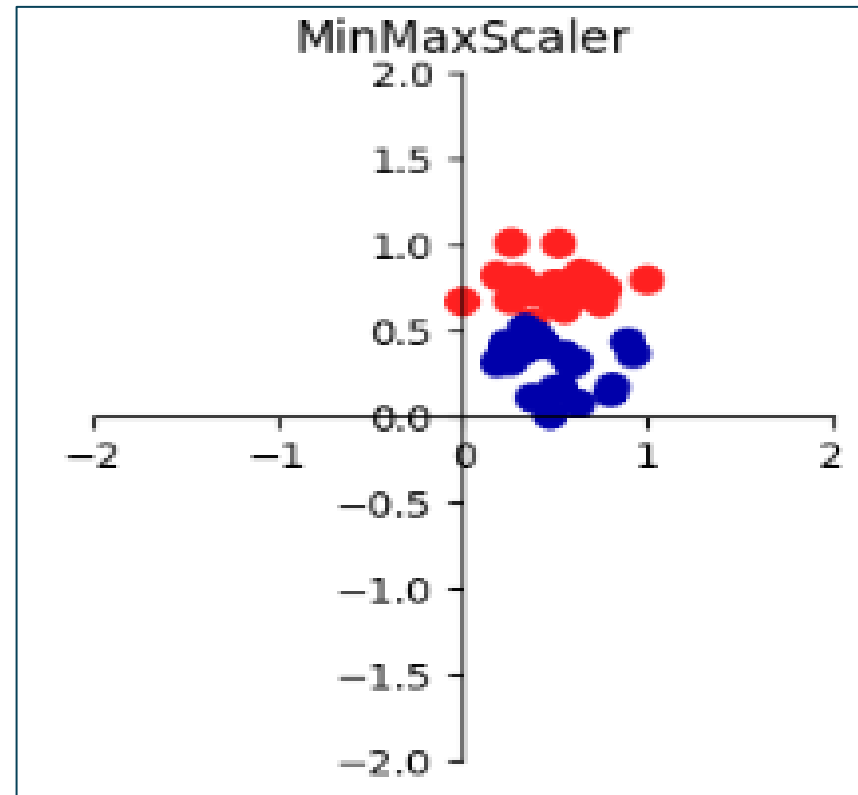
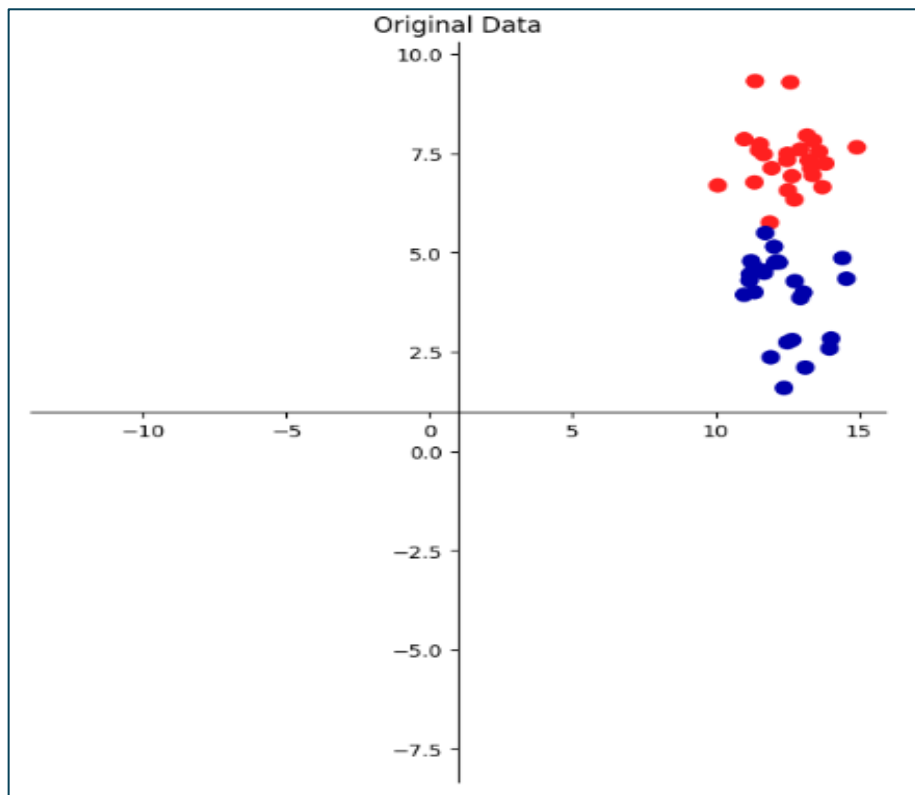
- ❖ The second highest 25% of numbers.
- ❖ The highest 25% of numbers.

RobustScaler



MinMaxScaler

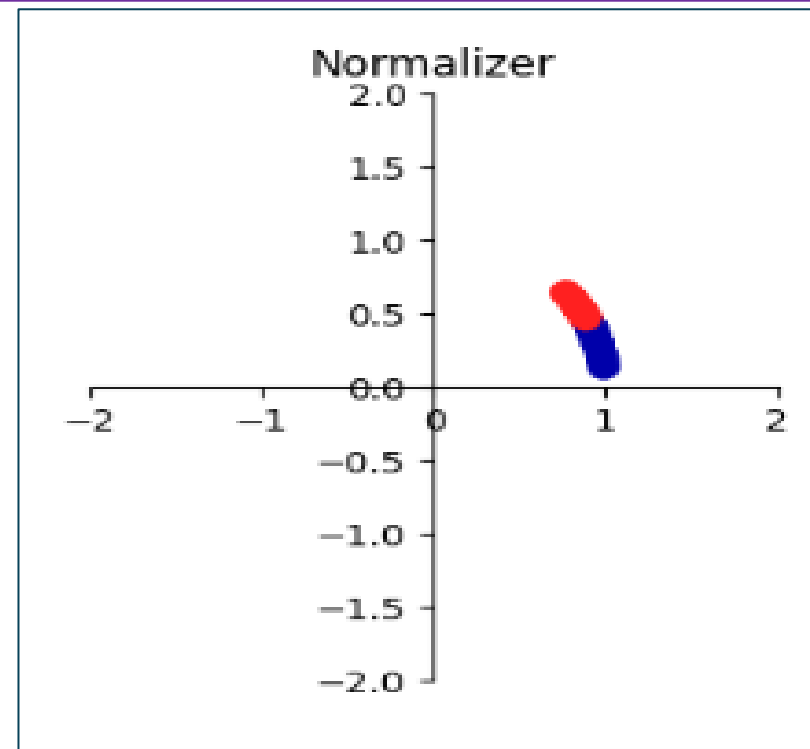
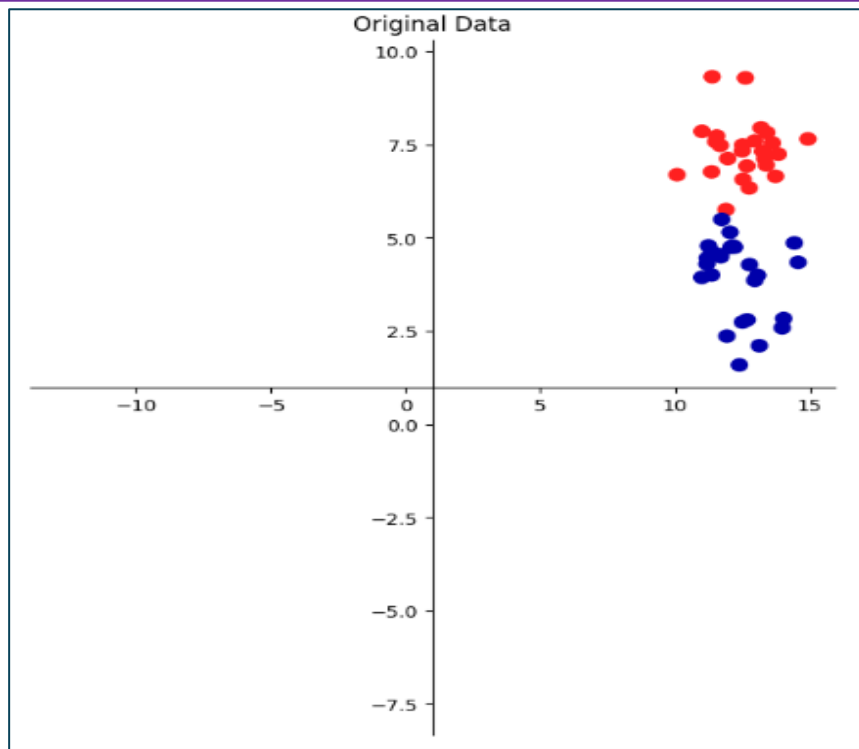
- ❖ The MinMaxScaler **shifts** the data such that all features are exactly between **0 and 1**.
- ❖ For two-dimensional dataset this mean **all of the data** is contained **within** the **rectangle** created by **the x-axis between 0 and 1 and the y-axis between 0 and 1**.



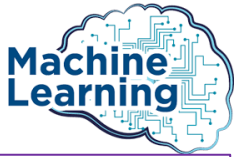
```
X_std = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))  
X_scaled = X_std * (max - min) + min
```

Normalizer

- ❖ The Normalizer does a very different kind of rescaling. It scale each data point such that the **feature vector has a Euclidean length of 1**. It projects a data point on the circle with a **radius of 1**. This means every data point is scaled by a different number (by the **inversed** of its length). This normalization is often used when only the direction (or angle) of the data matters, not the length of the feature vector.



Applying Data Transformation



- ❖ Preprocessing methods like the scalers are usually applied before applying a supervised machine learning algorithm.
- ❖ As an example, apply the kernel SVM to the cancer dataset, and use MinMaxScaler for preprocessing the data.

```
[2]: from sklearn.datasets import load_breast_cancer
      from sklearn.model_selection import train_test_split

      cancer=load_breast_cancer()

      X_train, X_test, y_train_, y_test=train_test_split(cancer.data, cancer.target, random_state=1)

      print(X_train.shape)
      print(X_test.shape)

      (426, 30)
      (143, 30)
```

MinMaxScaler

- ❖ In MinMaxScaler, the fit method computes the minimum and maximum value of each feature on the training set. The scaler is **only provided** with the data (**X_train**) when fit is called, and **y_train** is **not used**.

```
from sklearn.preprocessing import MinMaxScaler  
  
scaler=MinMaxScaler()  
scaler.fit(X_train)
```

[3]:

▼ MinMaxScaler ⓘ ?
MinMaxScaler()

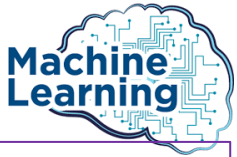
MinMaxScaler Transformation on Training Data (X_train)



```
[6]: X_train_scaled=scaler.transform(X_train)
print("transformed shape: {}".format(X_train_scaled.shape))
print("per-feature minimum before scaling:\n {}".format(X_train.min(axis=0)))
print("per-feature maximum before scaling:\n {}".format(X_train.max(axis=0)))
print("per-feature minimum after scaling:\n {}".format(X_train_scaled.min(axis=0)))
print("per-feature maximum before scaling:\n {}".format(X_train_scaled.max(axis=0)))
```

```
transformed shape: (426, 30)
per-feature minimum before scaling:
[6.981e+00 9.710e+00 4.379e+01 1.435e+02 5.263e-02 1.938e-02 0.000e+00
 0.000e+00 1.060e-01 5.024e-02 1.153e-01 3.602e-01 7.570e-01 6.802e+00
 1.713e-03 2.252e-03 0.000e+00 0.000e+00 9.539e-03 8.948e-04 7.930e+00
 1.202e+01 5.041e+01 1.852e+02 7.117e-02 2.729e-02 0.000e+00 0.000e+00
 1.566e-01 5.521e-02]
per-feature maximum before scaling:
[2.811e+01 3.928e+01 1.885e+02 2.501e+03 1.634e-01 2.867e-01 4.268e-01
 2.012e-01 3.040e-01 9.575e-02 2.873e+00 4.885e+00 2.198e+01 5.422e+02
 3.113e-02 1.354e-01 3.960e-01 5.279e-02 6.146e-02 2.984e-02 3.604e+01
 4.954e+01 2.512e+02 4.254e+03 2.226e-01 9.379e-01 1.170e+00 2.910e-01
 5.774e-01 1.486e-01]
per-feature minimum after scaling:
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0.]
per-feature maximum before scaling:
[1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1.]
```

MinMaxScaler Transformation on Testing Data (X_test)



- ❖ The transformed data has the same shape as the original data, the features are simply shifted and scaled. All of the features are between 0 and 1.
- ❖ To apply the SVM on the scaled data, need to transform the test set. It is done by calling the transform method on X_test.
- ❖ After scaling, the test set minimum and maximum are not 0 and 1. Some of the features are even outside the 0-1 range.
- ❖ MinMaxScaler always applies exactly the same transformation to the training and the test set.
- ❖ This means the transform method always subtracts the training set minimum and divides by the training set range, which might be different from the minimum and range for the test set.

MinMaxScaler Transformation on Testing Data (X_test)



```
[7]: X_test_scaled=scaler.transform(X_test)
print("per-feature minimum after scaling:\n{}".format(X_test_scaled.min(axis=0)))
print("per-feature maximum after scaling:\n{}".format(X_test_scaled.max(axis=0)))
```

per-feature minimum after scaling:

```
[ 0.0336031  0.0226581  0.03144219  0.01141039  0.14128374  0.04406704
  0.          0.          0.1540404 -0.00615249 -0.00137796  0.00594501
  0.00430665  0.00079567  0.03919502  0.0112206   0.          0.
 -0.03191387  0.00664013  0.02660975  0.05810235  0.02031974  0.00943767
  0.1094235   0.02637792  0.          0.          -0.00023764 -0.00182032]
```

per-feature maximum after scaling:

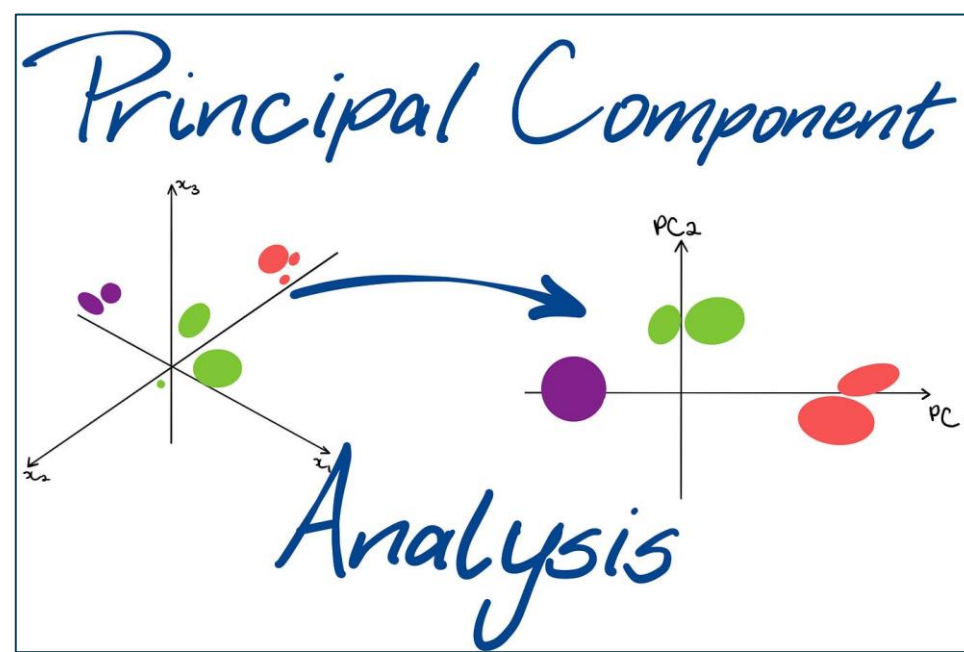
```
[0.9578778  0.81501522 0.95577362 0.89353128 0.81132075 1.21958701
 0.87956888 0.9333996  0.93232323 1.0371347  0.42669616 0.49765736
 0.44117231 0.28371044 0.48703131 0.73863671 0.76717172 0.62928585
 1.33685792 0.39057253 0.89612238 0.79317697 0.84859804 0.74488793
 0.9154725  1.13188961 1.07008547 0.92371134 1.20532319 1.63068851]
```

Dimensionality Reduction, Feature Extraction, and Manifold Learning

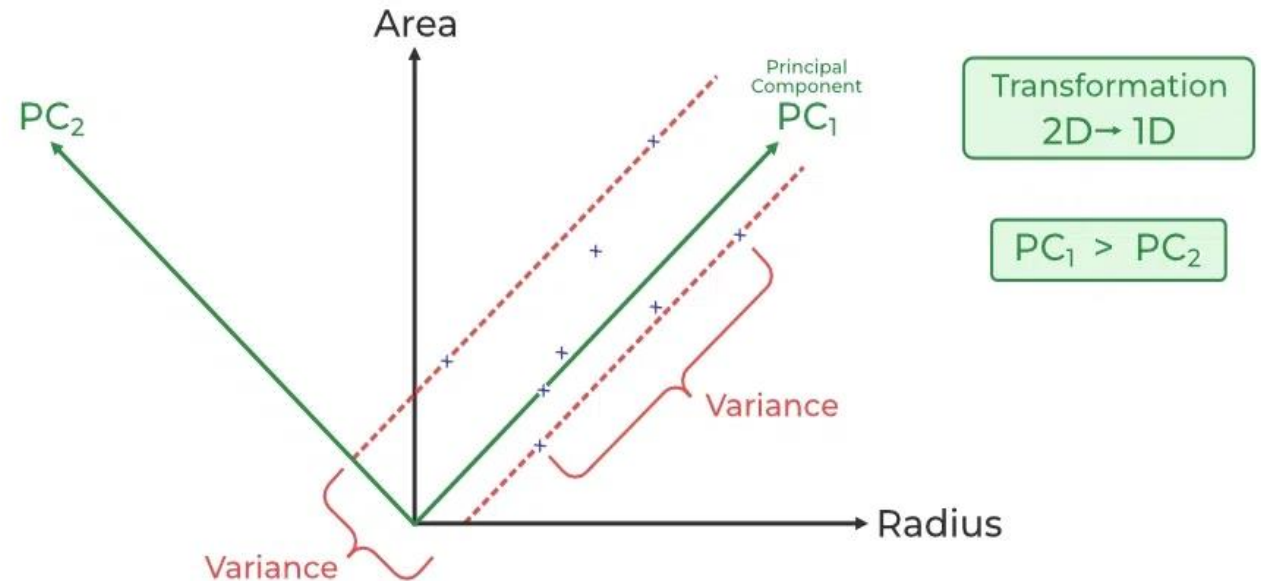


- ❖ The most common motivations of unsupervised learning are **visualization**, **compressing** the data, and finding a **representation** that is more informative for further processing.
- ❖ One of the simplest and most widely used algorithms for all of these is **principal component analysis (PCA)**.
- ❖ Two other algorithms are: **non-negative matrix factorization (NMF)**, which is commonly used for **feature extraction**, and **t-SNE**, which is commonly used for **visualization** using two-dimensional scatter plots.

❖ Principal component analysis is a method that **rotates** the **dataset** in a way such that the **rotated features are statistically uncorrelated**.

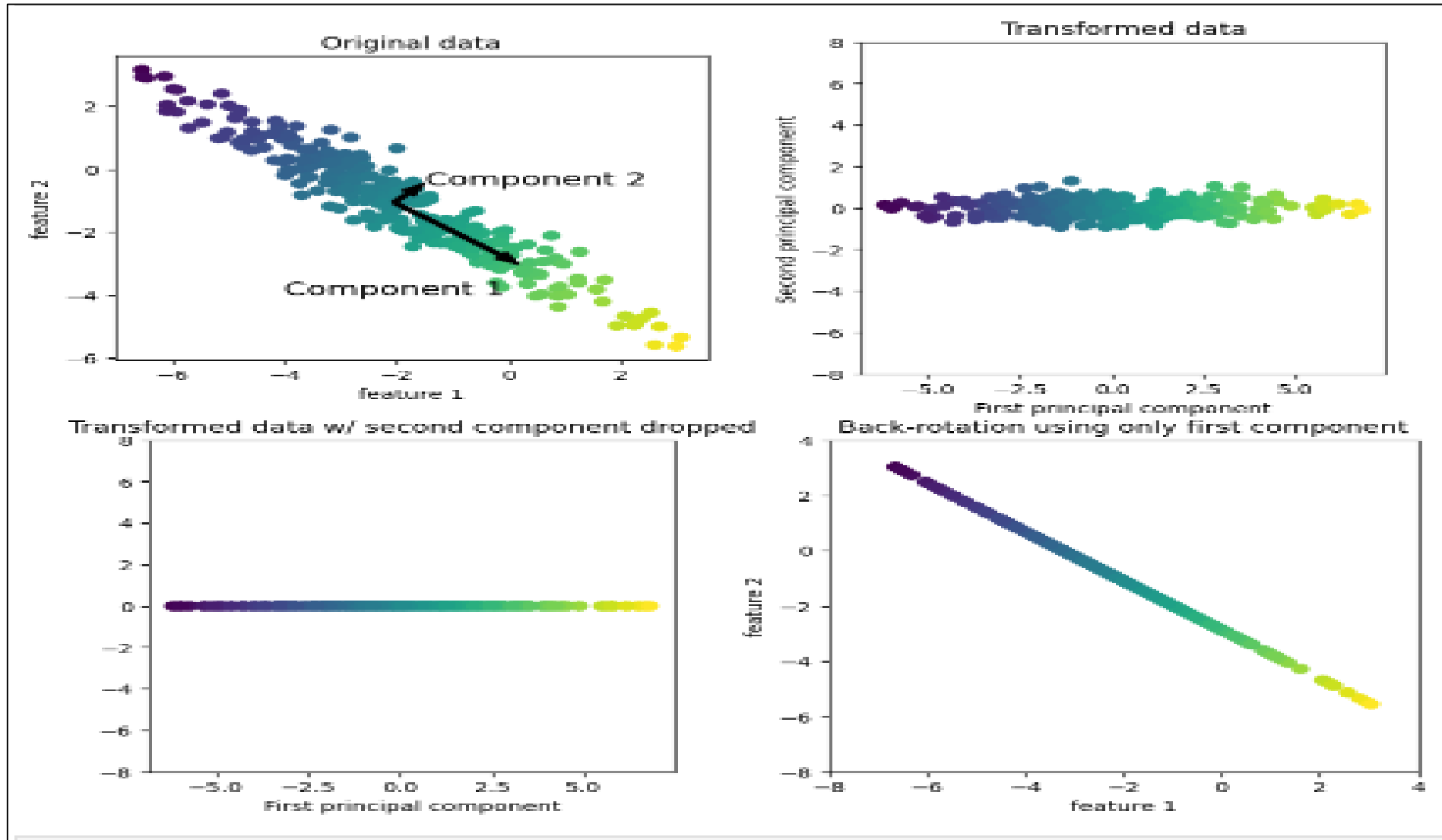


❖ This rotation is often followed by selecting only a subset of the new features, according to how important for explaining data.

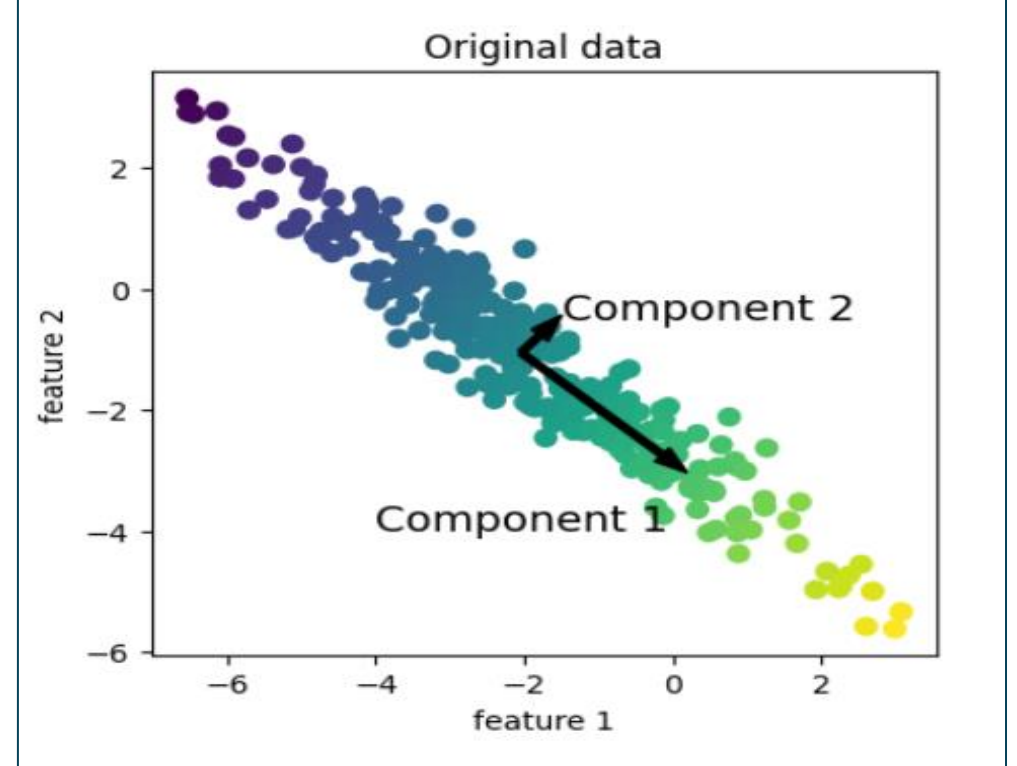


```
import mglearn
mglearn.plots.plot_pca_illustration()
```

The following example illustrates the effect of PCA on synthetic two-dimensional dataset.

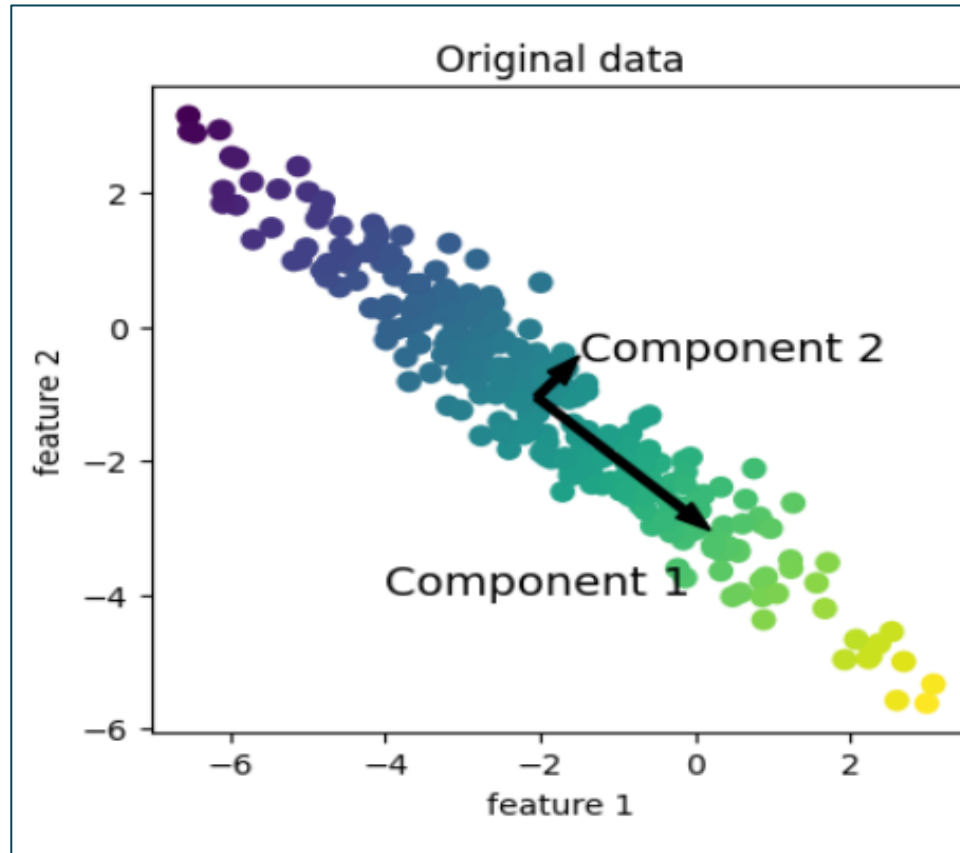


The first plot shows the original data points, colored to distinguish among them. The algorithm proceeds by first finding the **direction of maximum variance**, labeled “**Component 1**”. This is the direction (**vector**) in the data that contains most of the information, the direction along which the **features are most correlated with each other**.

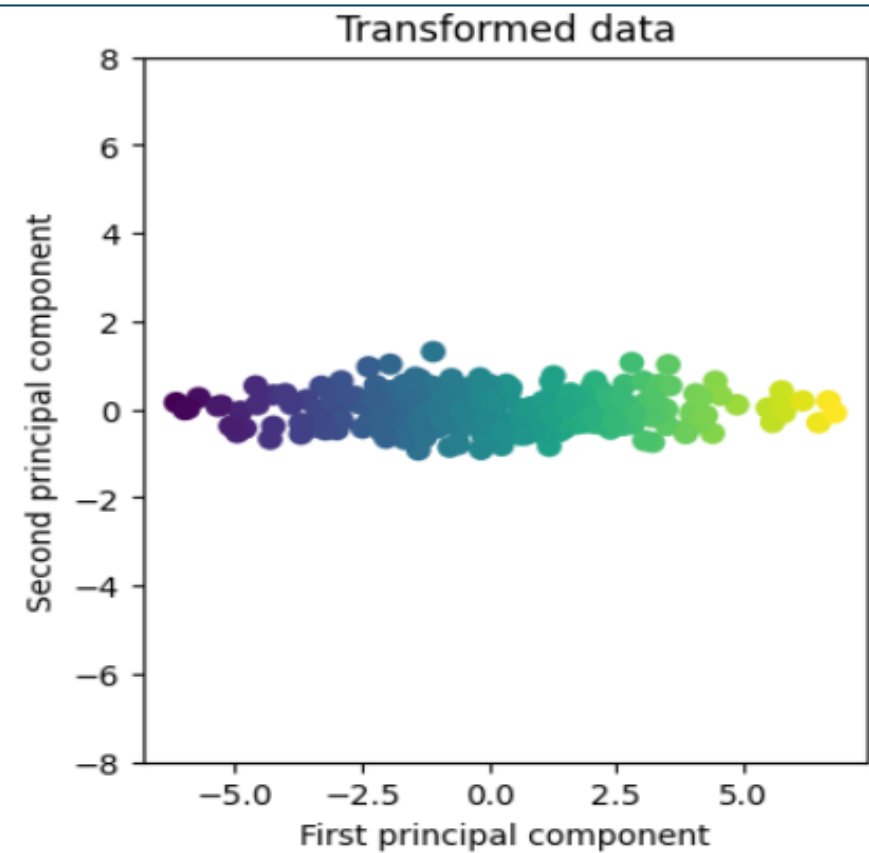


Then the algorithm finds the direction that contains the most information while being **orthogonal** (at the right angle) to the **first direction**. In two dimensions, there is **only one possible orientation** that is at a right angle, but in higher-dimensional spaces there would be (infinitely) many orthogonal directions. Although the **two components are drawn as arrows**, it doesn't really matter where the head and the tail are; we could drawn the first component from the center up to the top left instead of down to the bottom right. **The directions found using this process are called principal components**, as they are the main **direction of variance in the data**. In general, there are as **many principal components as original features**.

First Panel

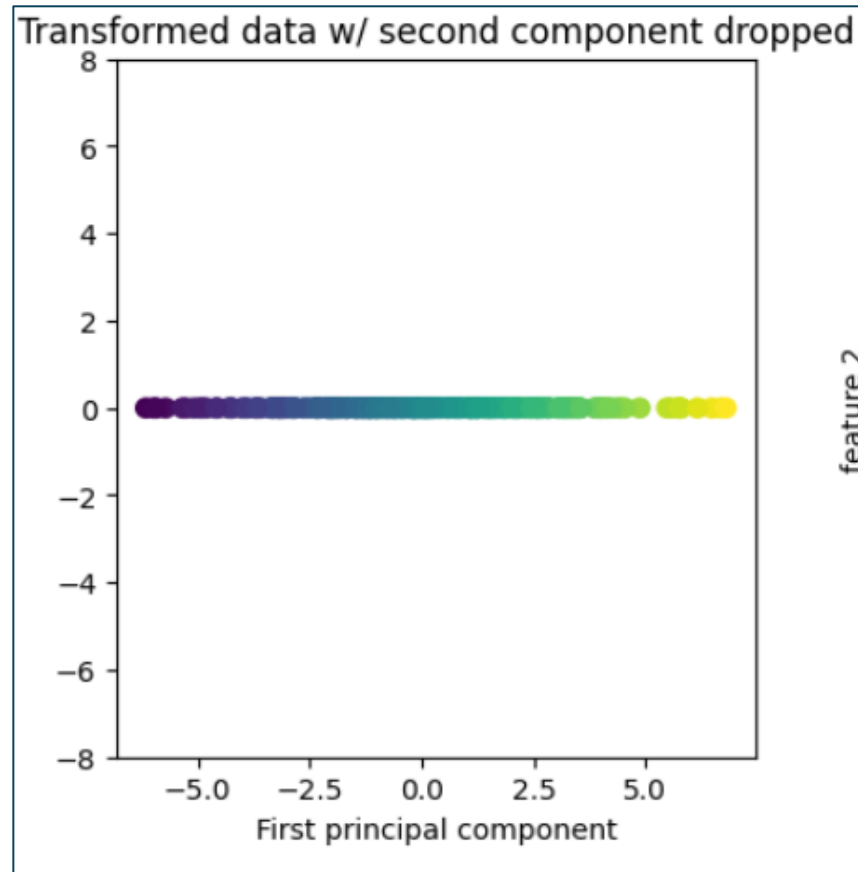


Second Panel

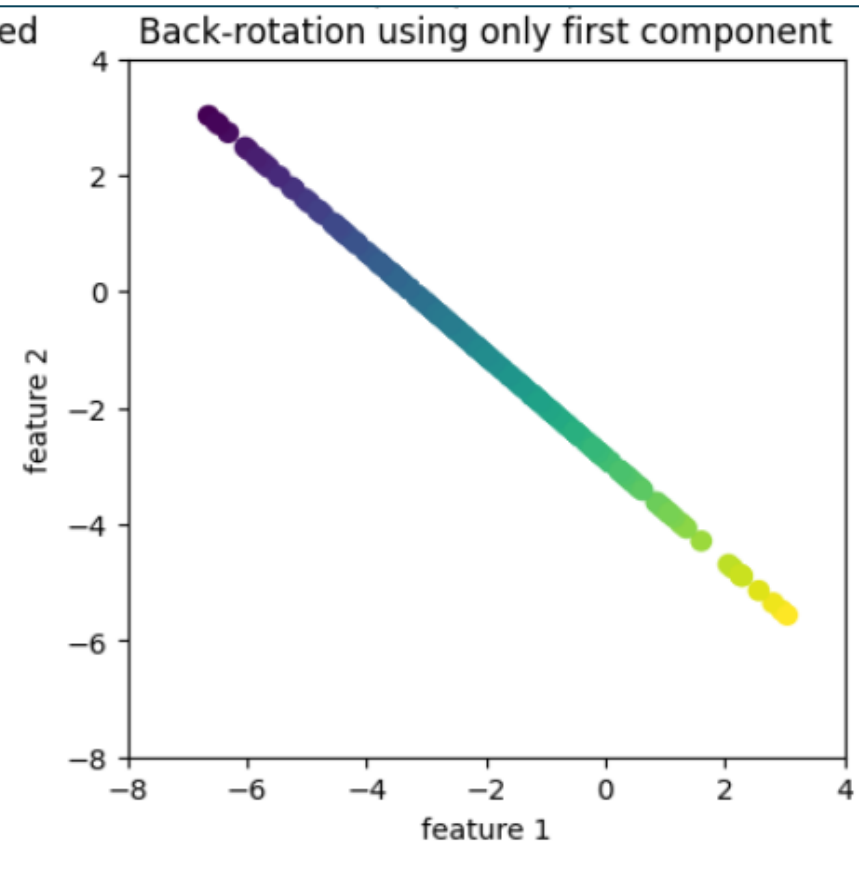


The second plot (top right) shows the **same data**, but now **rotated** so that the **first principal component aligns with the x-axis** and the **second principal component aligns with the y-axis**. Before the rotation, the **mean was subtracted from the data**, so that the transformed data is **centered around zero**. In the rotated representation found by PCA, the **two axes are uncorrelated**, meaning that the **correlation matrix of the data in this representation is zero except for the diagonal**.

Third Panel

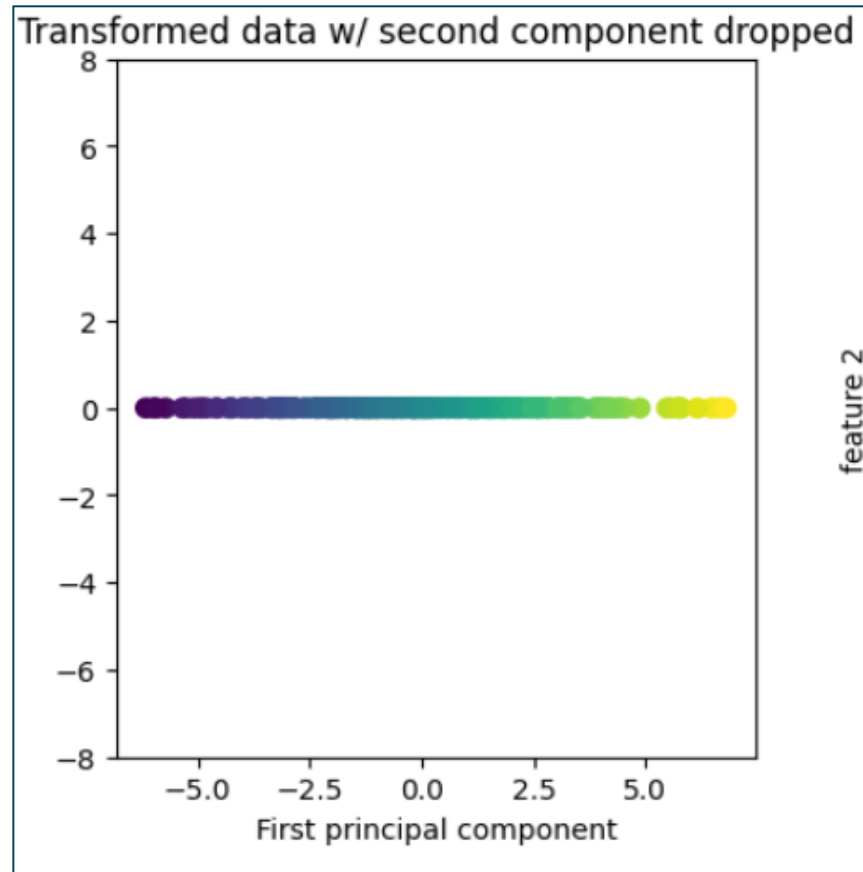


Last Panel

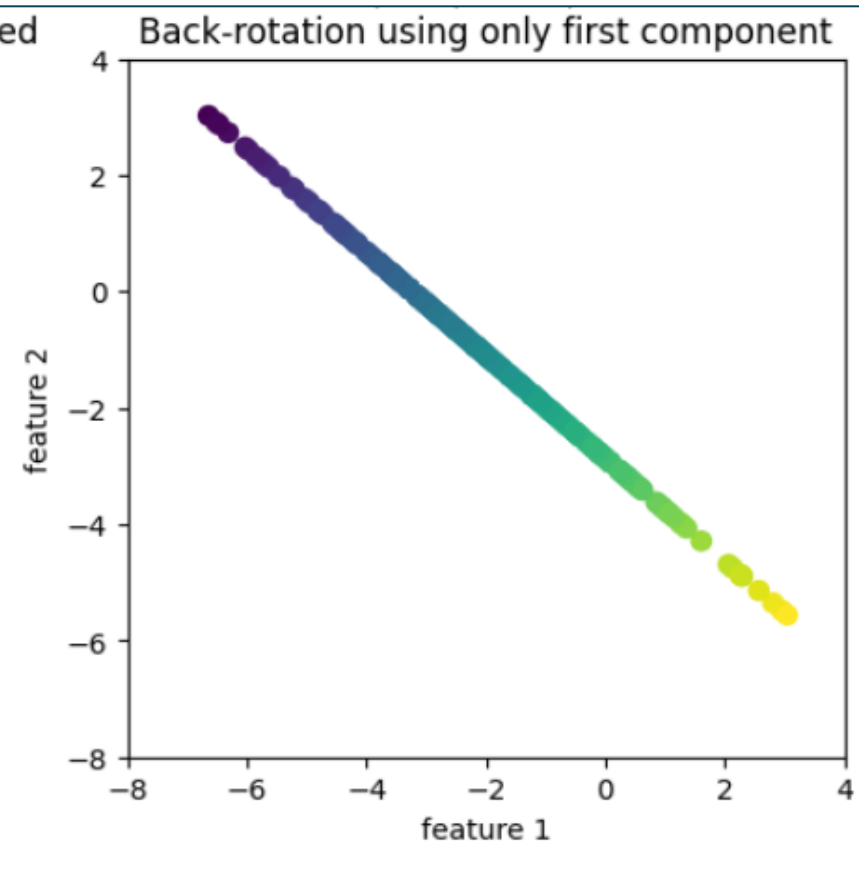


We can use **PCA for dimensionality reduction** by retaining only some of the principal components. In this example, we might keep only the first principal component, as shown in the **third panel**. This **reduces the data from two-dimensional dataset to a one-dimensional dataset**. However, instead of keeping only one of the original features, we found the most interesting direction (top left to bottom right in first panel) and kept this direction, the first principal component.

Third Panel



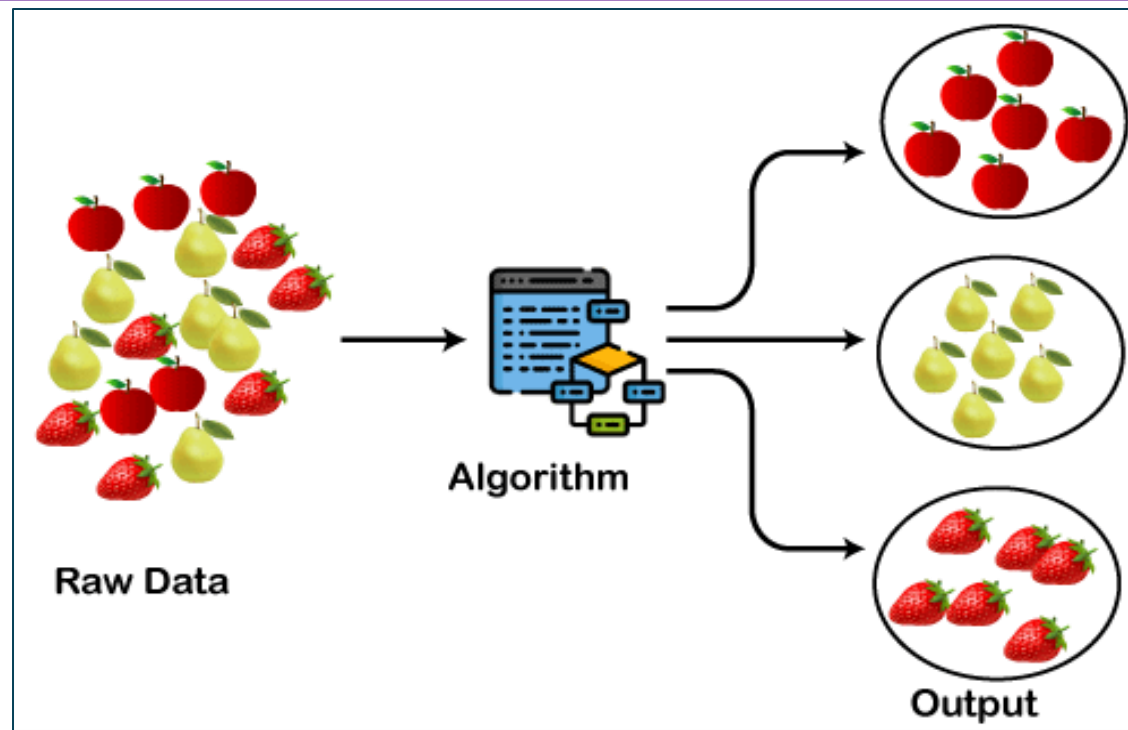
Last Panel



Finally, **undo the rotation** and **add the mean back to the data**. This will result in the data shown in the last panel. These points are in the **original feature space**, but keep only the information contained in the first principal component. This transformation is sometimes **used to remove noise effects from the data** or **visualize what part of the information is retained using the principal components**.

Clustering

- ❖ Clustering is the task of partitioning the **dataset into groups**, called **clusters**.
- ❖ The goal is to split up the data in such a way that point within a single cluster are very similar and points in different clusters are different.
- ❖ Similarly to classification algorithms, clustering algorithms assign (or predict) a number to each data point, indicating which cluster a particular point belongs to.



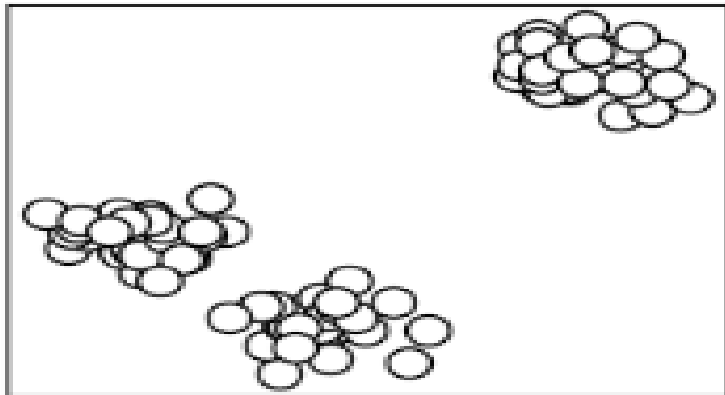
K-Means Clustering



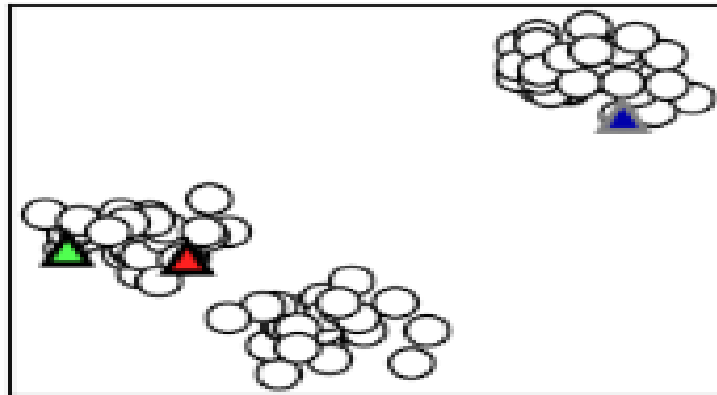
- ❖ K-means clustering is one of the simplest and most commonly used clustering algorithms. It tries to find cluster centers that are representative of certain regions of the data.
- ❖ The algorithm alternates between two steps: assigning each data point to the closet cluster center, and then setting each cluster center as the mean of the data points that are assigned to it.
- ❖ The algorithm is finished when the assignment of instances to clusters no longer changes.

```
import mglearn  
mglearn.plots.plot_kmeans_algorithm()
```

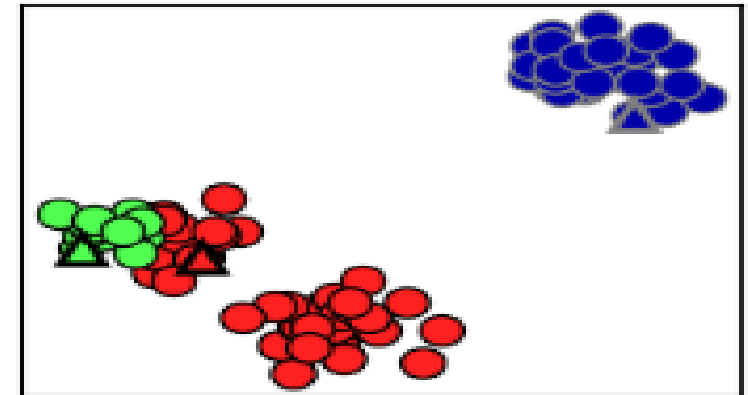
Input data



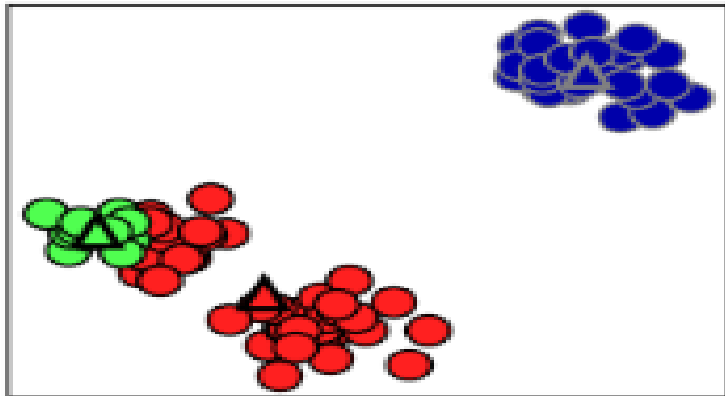
Initialization



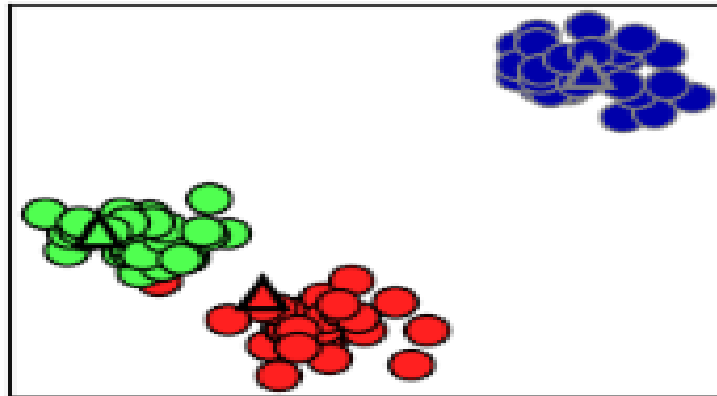
Assign Points (1)



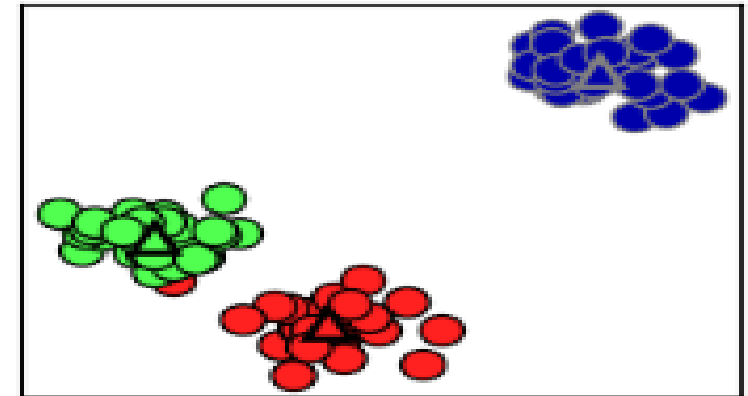
Recompute Centers (1)



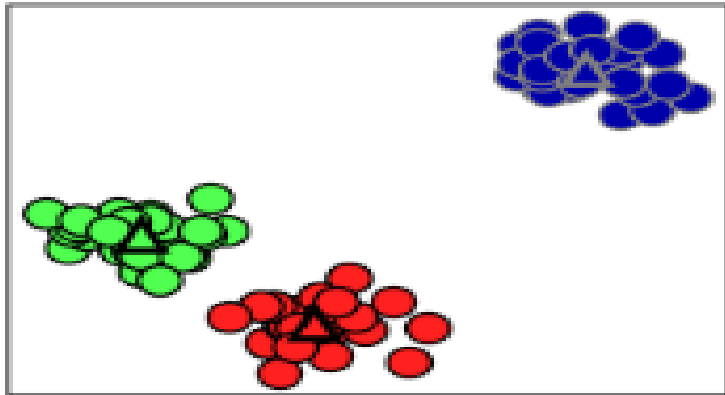
Reassign Points (2)



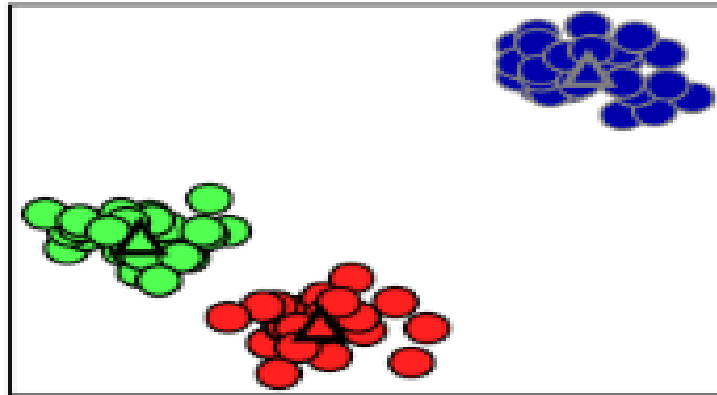
Recompute Centers (2)



Reassign Points (3)



Recompute Centers (3)



K-Means Clustering Steps

- Step 1: Choose the number of **cluster (K)**
- Step 2: Randomly select data points as **initial centroids**
- Step 3: **Assign datapoints to the nearest centroid** using **Distance Function**
- Step 4: Update Centroids
- Step 5: Repeat Step 3 and 4
- Step 6: Final output: Once **centroid stabilize**, K clusters with data points

Euclidean Distance Function

$$D_i(x_i, c_j) = \sqrt{(x_i - c_j)^2 + \dots + (x_n - c_j)^2}$$

D_i = Distance between data point and centroid

x_i = Data point

c_j = Centroid of cluster j

Update Centroids Function

$$c_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

C_j = set of data points assigned to cluster j

$|C_j|$ = number of points in cluster j

K-Means Clustering Sample Calculation

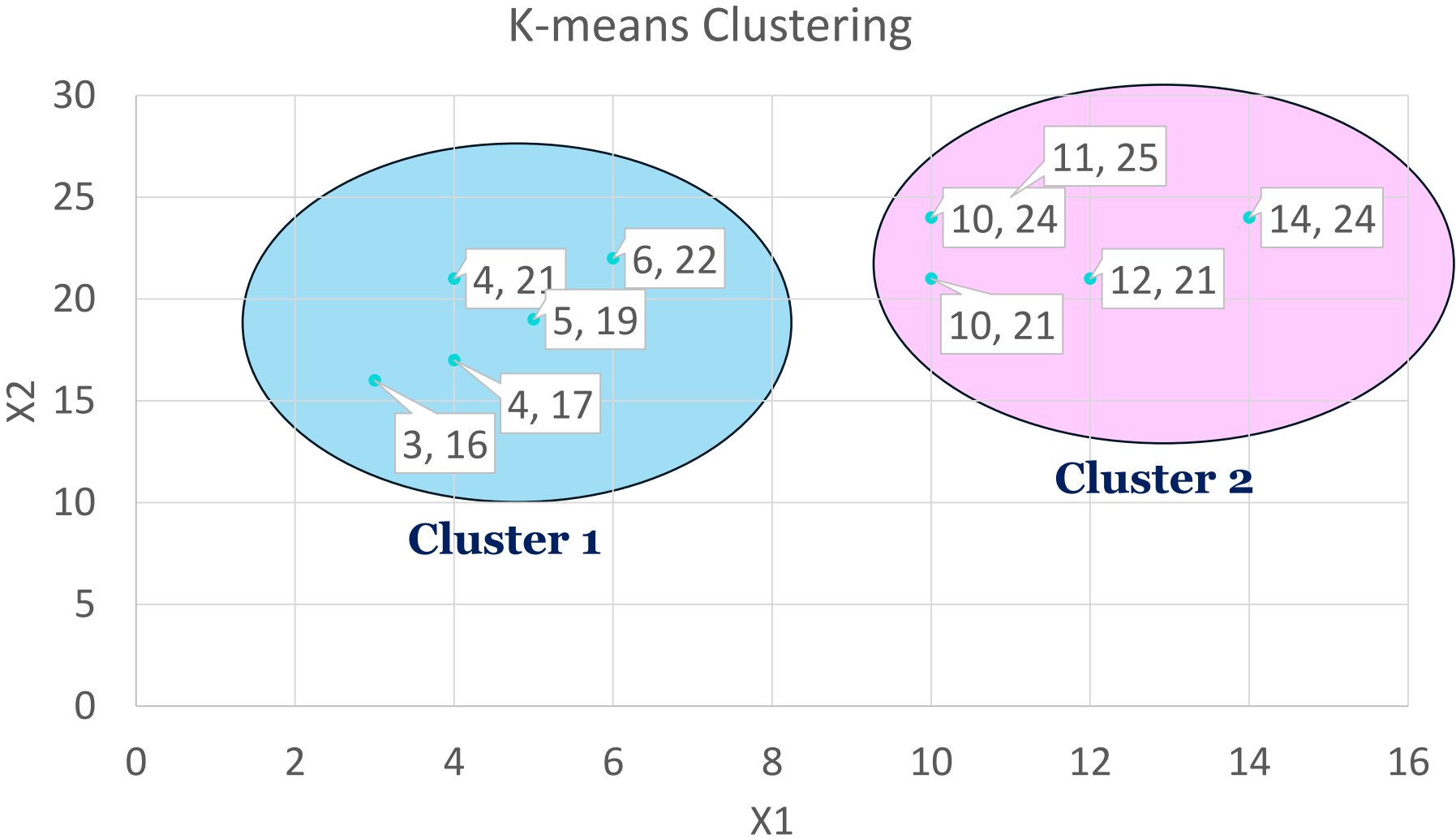
Iteration-1											
C1= (4,17)		C2=(14,24)					$D_1 = \sqrt{(x_i - c1_i)^2 + (x_j - c1_j)^2}$				
X		C1		C2		Distance1	Distance2	Closet	Update Centroid		
X1	X2	C1 _i	C1 _j	C2 _i	C2 _j				Centroid _i	Centroid _j	
4	21	4	17	14	24	4.00	10.44	1	4.4	19	Cluster-1
5	19	4	17	14	24	2.24	10.30	1	11.4	23	Cluster-2
10	24	4	17	14	24	9.22	4.00	2			
4	17	4	17	14	24	0.00	12.21	1			
3	16	4	17	14	24	1.41	13.60	1			
11	25	4	17	14	24	10.63	3.16	2			
14	24	4	17	14	24	12.21	0.00	2			
6	22	4	17	14	24	5.39	8.25	1			
10	21	4	17	14	24	7.21	5.00	2			
12	21	4	17	14	24	8.94	3.61	2			

K-Means Clustering Sample Calculation

Iteration-2		$D_1 = \sqrt{(x_i - c1_i)^2 + (x_j - c1_j)^2}$									
C1= (4.4, 19)		C2=(11.4, 23)									
X		C1		C2		Distance1	Distance2	Closet	Update Centroid		
X1	X2	C1 _i	C1 _j	C2 _i	C2 _j				Centroid _i	Centroid _j	
4	21	4.4	19	11.4	23	2.04	7.67	1	4.4	19	Cluster-1
5	19	4.4	19	11.4	23	0.60	7.55	1	11.4	23	Cluster-2
10	24	4.4	19	11.4	23	7.51	1.72	2			
4	17	4.4	19	11.4	23	2.04	9.53	1			
3	16	4.4	19	11.4	23	3.31	10.93	1			
11	25	4.4	19	11.4	23	8.92	2.04	2			
14	24	4.4	19	11.4	23	10.82	2.79	2			
6	22	4.4	19	11.4	23	3.40	5.49	1			
10	21	4.4	19	11.4	23	5.95	2.44	2			
12	21	4.4	19	11.4	23	7.86	2.09	2			

K-Means Clustering Sample Calculation: Final Output

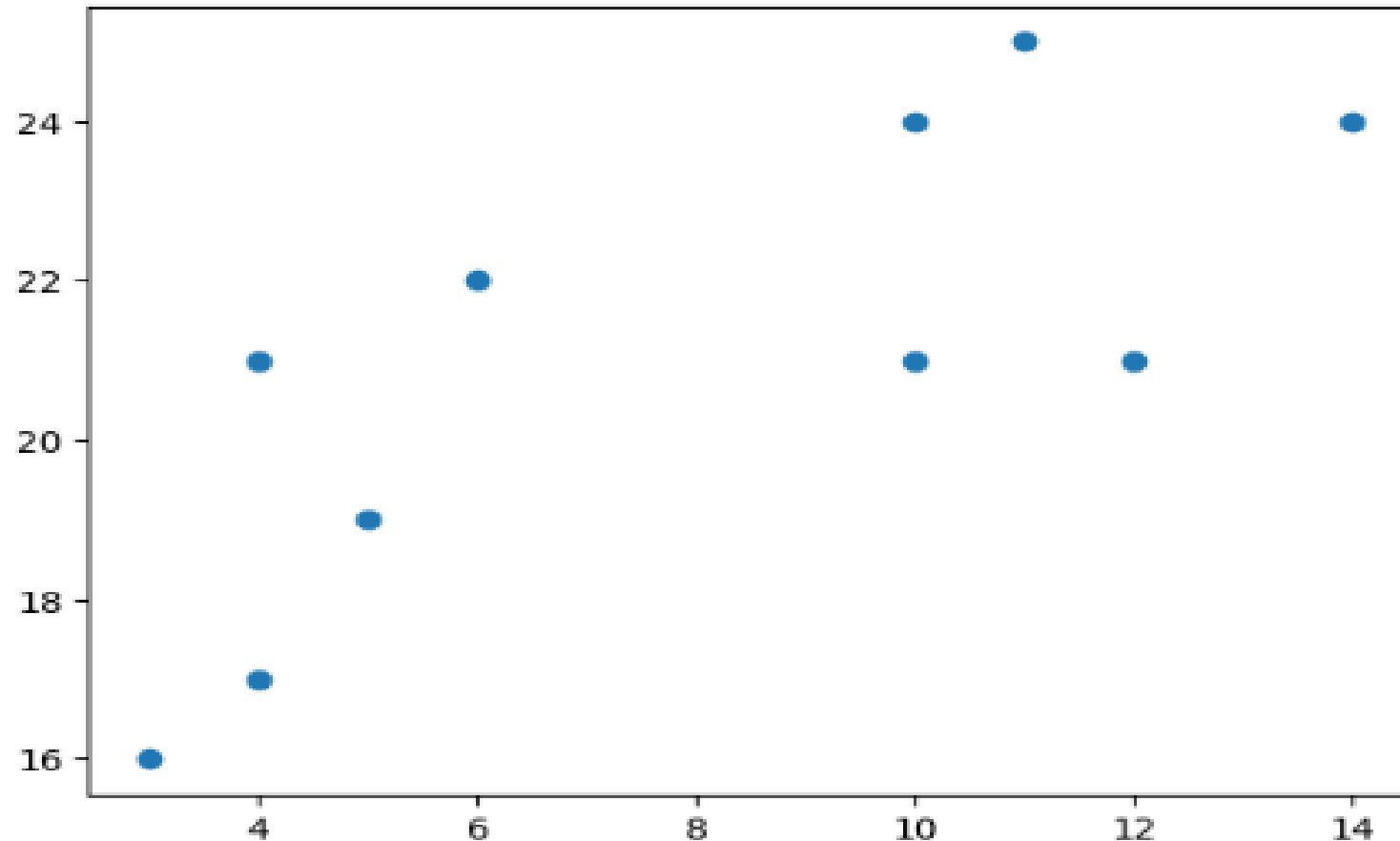
X	
X1	X2
4	21
5	19
10	24
4	17
3	16
11	25
14	24
6	22
10	21
12	21



K-Means Clustering Code Example

```
import matplotlib.pyplot as plt
x1=[4,5,10,4,3,11,14,6,10,12]
x2=[21,19,24,17,16,25,24,22,21,21]

#data=np.array([[4, 21], [5, 19], [10, 24], [4, 17], [3, 16], [11, 25], [14, 24], [6, 22], [10, 21], [12, 21]])
plt.scatter(x1,x2)
plt.show()
```



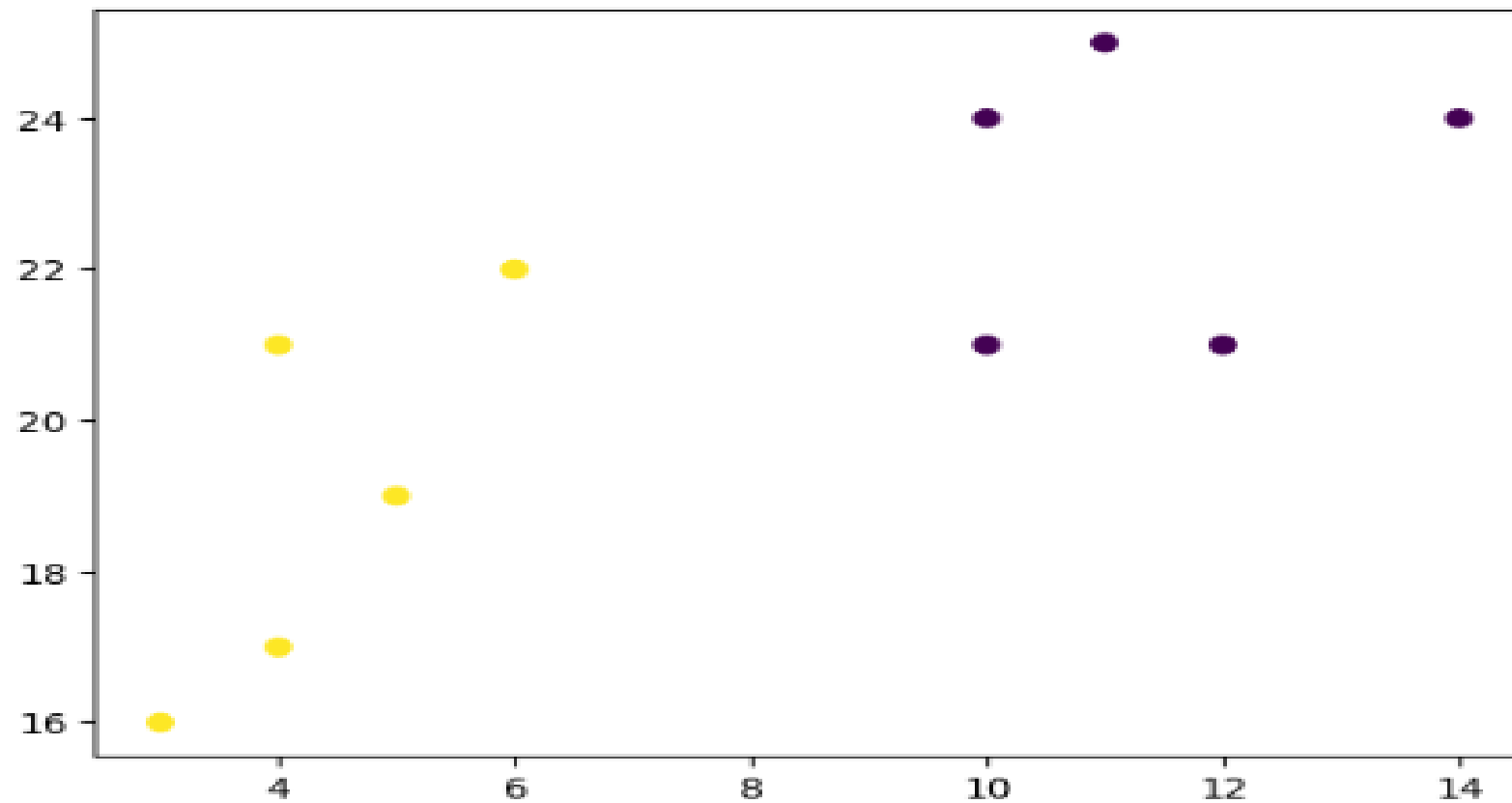
K-Means Clustering Code Example

```
from sklearn.cluster import KMeans

data=list(zip(x1,x2))

kmeans=KMeans(n_clusters=2)
kmeans.fit(data)

plt.scatter(x1,x2, c=kmeans.labels_)
plt.show()
```



References

Introduction to Machine Learning with Python: A Guide for Data Scientists, Andreas C. Müller & Sarah Guido.

Code Example

https://github.com/amueller/introduction_to_ml_with_python

Elementary Statistics: A Step by Step Approach, 9th Edition,
Allan G. Bluman

